Capítulo 2

Estrutura de um SO

Este capítulo apresenta os principais componentes de um sistema operacional e os mecanismos de hardware necessários para sua implementação.

2.1 Elementos do sistema operacional

Um sistema operacional não é um bloco único e fechado de software executando sobre o hardware. Na verdade, ele é composto de diversos componentes com objetivos e funcionalidades complementares. Alguns dos componentes mais relevantes de um sistema operacional típico são:

Núcleo: é o coração do sistema operacional, responsável pela gerência dos recursos do hardware usados pelas aplicações. Ele também implementa as principais abstrações utilizadas pelos aplicativos e programas utilitários.

Código de inicialização: (*boot code*) a inicialização do hardware requer uma série de tarefas complexas, como reconhecer os dispositivos instalados, testá-los e configurá-los adequadamente para seu uso posterior. Outra tarefa importante é carregar o núcleo do sistema operacional em memória e iniciar sua execução.

Drivers: módulos de código específicos para acessar os dispositivos físicos. Existe um driver para cada tipo de dispositivo, como discos rígidos SATA, portas USB, placas gráfica, etc. Muitas vezes o *driver* é construído pelo próprio fabricante do hardware e fornecido em forma compilada (em linguagem de máquina) para ser acoplado ao restante do sistema operacional.

Programas utilitários: são programas que facilitam o uso do sistema computacional, fornecendo funcionalidades complementares ao núcleo, como formatação de discos e mídias, configuração de dispositivos, manipulação de arquivos (mover, copiar, apagar), interpretador de comandos, terminal, interface gráfica, gerência de janelas, etc.

As diversas partes do sistema operacional estão relacionadas entre si conforme apresentado na Figura 2.1. Na figura, a região cinza indica o sistema operacional propriamente dito. A forma como os diversos componentes do SO são interligados e se relacionam

varia de sistema para sistema; algumas possibilidades de organização são discutidas no Capítulo 3.

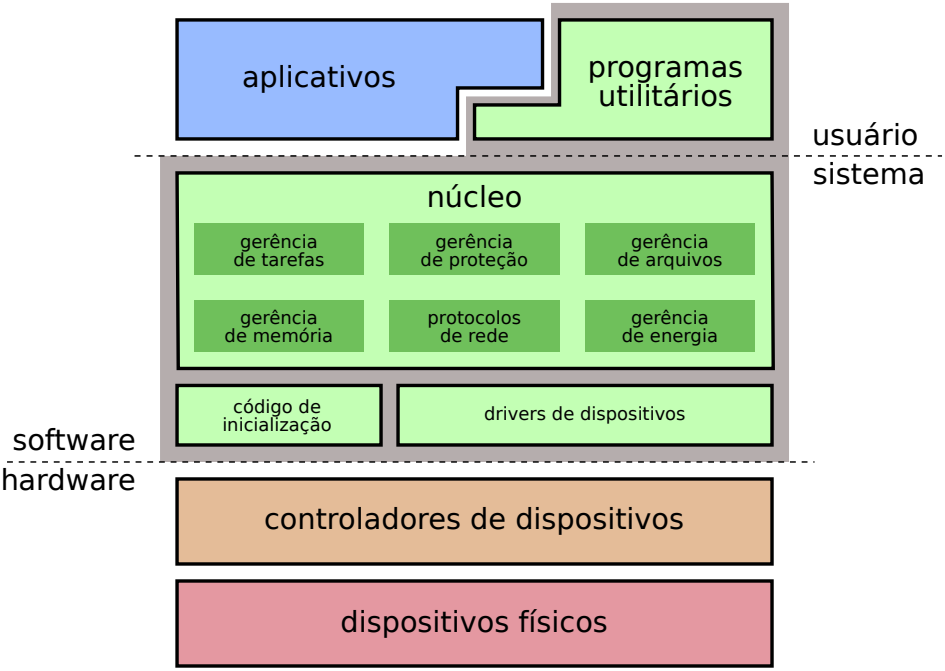


Figura 2.1: Estrutura de um sistema operacional típico

A camada mais baixa do sistema operacional, que constitui o chamado "núcleo" do sistema (ou (*kernel*), usualmente executa em um modo especial de operação do processador, denominado *modo privilegiado* ou modo sistema (vide Seção 2.2.3). Os demais programas e aplicações executam em um modo denominado *modo usuário*.

Sistemas operacionais reais têm estruturas que seguem aquela apresentada na Figura 2.1, embora sejam geralmente muito mais complexas. Por exemplo, o sistema operacional Android, usado em telefones celulares, é baseado no Linux, mas tem uma rica estrutura de bibliotecas e serviços no nível de usuário, que são usados para a construção de aplicações. A Figura 2.2 representa a arquitetura de um sistema Android 8.0.

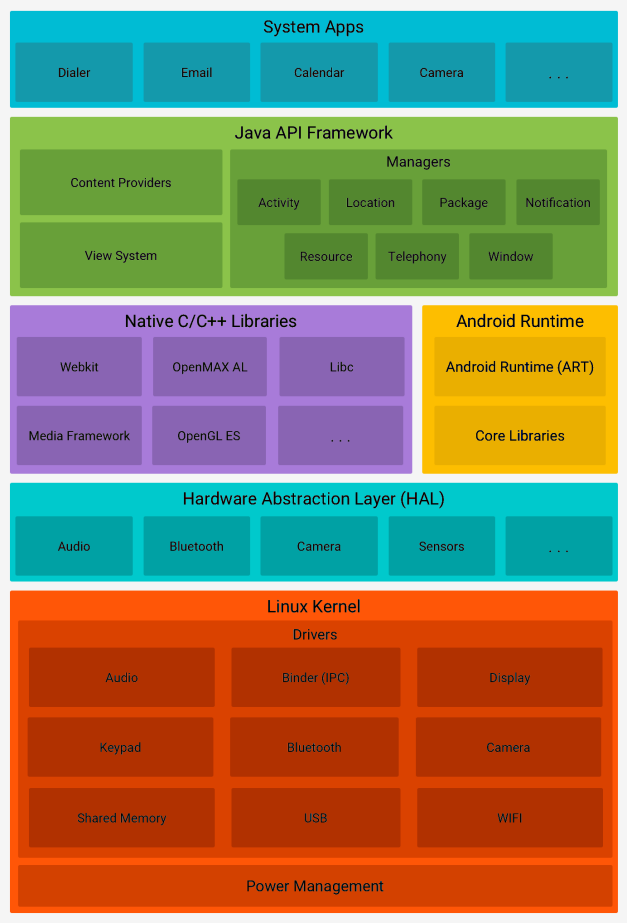


Figura 2.2: Estrutura de um sistema operacional Android [Google, 2018].

2.2 Elementos de hardware

O sistema operacional executa diretamente sobre o hardware, abstraindo e gerenciando recursos para as aplicações. Para que o SO possa cumprir suas funções com eficiência e confiabilidade, o hardware deve prover algumas funcionalidades básicas que são discutidas nesta seção.

2.2.1 Arquitetura do computador

Um computador típico é constituído de um ou mais processadores, responsáveis pela execução das instruções das aplicações, uma ou mais áreas de memórias que armazenam as aplicações em execução (seus códigos e dados) e dispositivos periféricos que

permitem o armazenamento de dados e a comunicação com o mundo exterior, como discos, terminais e teclados.

A maioria dos computadores monoprocessados atuais segue uma arquitetura básica definida nos anos 40 por János (John) Von Neumann, conhecida por "arquitetura Von Neumann". A principal característica desse modelo é a ideia de "programa armazenado", ou seja, o programa a ser executado reside na memória junto com os dados. Os principais elementos constituintes do computador estão interligados por um ou mais barramentos (para a transferência de dados, endereços e sinais de controle). A Figura 2.3 ilustra a arquitetura de um computador típico.

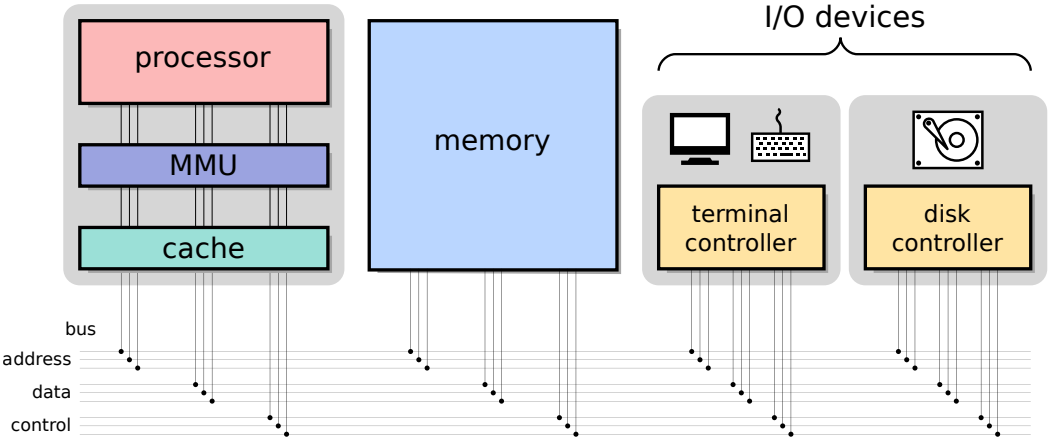


Figura 2.3: Arquitetura de um computador típico

O centro do sistema de computação é o processador. Ele é responsável por continuamente ler instruções e dados da memória ou dos periféricos, processá-los e enviar os resultados de volta à memória ou a outros periféricos. Em sua forma mais simples, um processador convencional é constituído de uma unidade lógica e aritmética (ULA), que realiza os cálculos e operações lógicas, um conjunto de registradores para armazenar dados de trabalho e alguns registradores para funções especiais (contador de programa, ponteiro de pilha, flags de status, etc.).

Processadores modernos são incrivelmente mais complexos, podendo possuir diversos núcleos de processamento (os chamados *cores*), cada um contendo vários processadores lógicos internos (*hyperthreading*). Além disso, um computador pode conter vários processadores trabalhando em paralelo. Outro aspecto diz respeito à memória, que pode comportar vários níveis de cache, dentro do processador e na placa-mãe. Uma descrição mais detalhada da arquitetura de computadores modernos pode ser encontrada em [Stallings, 2010].

Todas as transferências de dados entre processador, memória e periféricos são feitas através dos barramentos: o **barramento de endereços** indica a posição de memória (ou o dispositivo) a acessar, o **barramento de controle** indica a operação a efetuar (leitura ou escrita) e o **barramento de dados** transporta a informação a ser transferida entre o processador e a memória ou um controlador de dispositivo.

O acesso do processador à memória é mediado por um controlador específico (que pode estar fisicamente dentro do próprio chip do processador): a **Unidade de Gerência de Memória** (MMU - *Memory Management Unit*). Ela é responsável por analisar cada endereço de memória acessado pelo processador, validá-lo, efetuar conversões de endereçamento porventura necessárias e executar a operação solicitada pelo processador (leitura ou escrita de uma posição de memória). Uma memória *cache* permite armazenar os dados mais recentemente lidos da memória, para melhorar o desempenho.

Os periféricos do computador (discos, teclado, monitor, etc.) são acessados através de circuitos eletrônicos específicos, denominados **controladores**: a placa de vídeo permite o acesso ao monitor, a placa *ethernet* dá acesso à rede, o controlador USB permite acesso ao mouse, teclado e outros dispositivos USB externos. Dentro do computador, cada dispositivo é representado por seu respectivo controlador. Os controladores podem ser acessados através de *portas de entrada/saída* endereçáveis: a cada controlador é atribuída uma faixa de endereços de portas de entrada/saída. A Tabela 2.1 a seguir apresenta alguns endereços portas de entrada/saída para acessar controladores em um PC típico (podem variar):

Dispositivo	**Endereços de acesso**
temporizador	0040-0043
teclado	0060-006F
porta serial COM1	02F8-02FF
controlador SATA	30BC-30BF
controlador Ethernet	3080-309F
controlador	3000-303F

Tabela 2.1: Endereços de acesso a dispositivos (em hexadecimal).

2.2.2 Interrupções e exceções

A comunicação entre o processador e os dispositivos se dá através do acesso às portas de entrada/saída, que podem ser lidas e/ou escritas pelo processsador. Esse acesso é feito por iniciativa do processador, quando este precisa ler ou escrever dados no dispositivo. Entretanto, muitas vezes um dispositivo precisa informar o processador rapidamente sobre um evento interno, como a chegada de um pacote de rede, um clique de mouse ou a conclusão de uma operação de disco. Neste caso, o controlador tem duas alternativas:

- aguardar até que o processador o consulte, o que poderá ser demorado caso o processador esteja ocupado com outras tarefas (o que geralmente ocorre);
- notificar o processador, enviando a ele uma *requisição de interrupção* (IRQ – *Interrupt ReQuest*) através do barramento de controle.

Ao receber a requisição de interrupção, os circuitos do processador suspendem seu fluxo de execução corrente e desviam para um endereço predefinido, onde se encontra

uma *rotina de tratamento de interrupção* (*interrupt handler*). Essa rotina é responsável por tratar a interrupção, ou seja, executar as ações necessárias para atender o dispositivo que a gerou. Ao final da rotina de tratamento da interrupção, o processador retoma o código que estava executando quando recebeu a requisição.

A Figura 2.4 representa os principais passos associados ao tratamento de uma interrupção envolvendo a placa de rede Ethernet, enumerados a seguir:

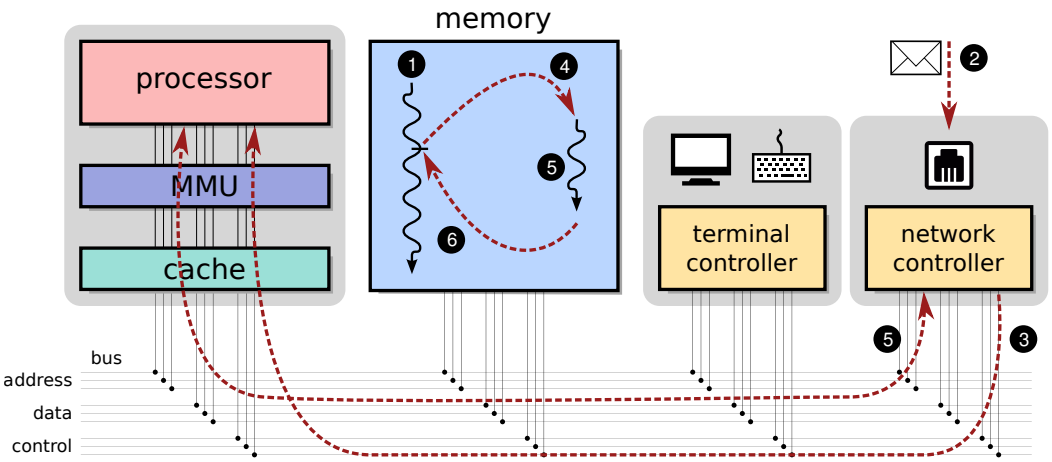


Figura 2.4: Roteiro típico de um tratamento de interrupção

1. o processador está executando um programa qualquer (em outras palavras, um fluxo de execução);
2. um pacote vindo da rede é recebido pela placa Ethernet;
3. o controlador Ethernet envia uma solicitação de interrupção (IRQ) ao processador;
4. o processamento é desviado do programa em execução para a rotina de tratamento da interrupção;
5. a rotina de tratamento é executada para interagir com o controlador de rede (via barramentos de dados e de endereços) para transferir os dados do pacote de rede do controlador para a memória;
6. a rotina de tratamento da interrupção é finalizada e o processador retorna à execução do programa que havia sido interrompido.

Essa sequência de ações ocorre a cada requisição de interrupção recebida pelo processador, que geralmente corresponde a um evento ocorrido em um dispositivo: a chegada de um pacote de rede, um click no mouse, uma operação concluída pelo disco, etc. Interrupções não são eventos raros, pelo contrário: centenas ou mesmo milhares de interrupções são recebidas pelo processador por segundo, dependendo da carga e da configuração do sistema (número e tipo dos periféricos). Assim, as rotinas de tratamento

de interrupção devem ser curtas e realizar suas tarefas rapidamente, para não prejudicar o desempenho do sistema.

Para distinguir interrupções geradas por dispositivos distintos, cada interrupção é identificada pelo hardware por um número inteiro. Como cada interrupção pode exigir um tipo de tratamento diferente (pois os dispositivos são diferentes), cada IRQ deve disparar sua própria rotina de tratamento de interrupção. A maioria das arquiteturas atuais define uma tabela de endereços de funções denominada *Tabela de Interrupções* (IVT - *Interrupt Vector Table*); cada entrada dessa tabela aponta para a rotina de tratamento da interrupção correspondente. Por exemplo, se a entrada 5 da tabela contém o valor `3C20h`, então a rotina de tratamento da IRQ 5 iniciará na posição `3C20h` da memória RAM. A tabela de interrupções reside em uma posição fixa da memória RAM, definida pelo fabricante do processador, ou tem sua posição indicada pelo conteúdo de um registrador da CPU específico para esse fim.

A maioria das interrupções recebidas pelo processador têm como origem eventos externos a ele, ocorridos nos dispositivos periféricos e reportados por seus controladores. Entretanto, alguns eventos gerados pelo próprio processador podem ocasionar o desvio da execução usando o mesmo mecanismo das interrupções: são as *exceções*. Ações como instruções ilegais (inexistentes ou com operandos inválidos), tentativas de divisão por zero ou outros erros de software disparam exceções no processador, que resultam na ativação de uma rotina de tratamento de exceção, usando o mesmo mecanismo das interrupções (e a mesma tabela de endereços de funções). A Tabela 2.2 representa parte da tabela de interrupções do processador Intel Pentium.

O mecanismo de interrupção torna eficiente a interação do processador com os dispositivos periféricos. Se não existissem interrupções, o processador perderia muito tempo "varrendo" todos os dispositivos do sistema para verificar se há eventos a serem tratados. Além disso, as interrupções permitem construir funções de entrada/saída assíncronas, ou seja, o processador não precisa esperar a conclusão de cada operação solicitada a um dispositivo, pois o dispositivo gera uma interrupção para "avisar" o processador quando a operação for concluída. O Capítulo 19 traz informações mais detalhadas sobre o tratamento de interrupções pelo sistema operacional.

2.2.3 Níveis de privilégio

Núcleo, *drivers*, utilitários e aplicações são constituídos basicamente de código de máquina. Todavia, devem ser diferenciados em sua capacidade de interagir com o hardware: enquanto o núcleo e os drivers devem ter pleno acesso ao hardware, para poder configurá-lo e gerenciá-lo, os aplicativos e utilitários devem ter acesso mais restrito a ele, para não interferir nas configurações e na gerência, o que poderia desestabilizar o sistema inteiro. Além disso, aplicações com acesso pleno ao hardware seriam um risco à segurança, pois poderiam contornar facilmente os mecanismos de controle de acesso aos recursos (tais como arquivos e áreas de memória).

Para permitir diferenciar os privilégios de execução dos diferentes tipos de software, os processadores modernos implementam *níveis de privilégio de execução*. Esses níveis são controlados por flags especiais na CPU, que podem ser ajustados em função do tipo de código em execução. Os processadores no padrão Intel x86, por exemplo, dispõem de 4 níveis de privilégio ($0 \ldots 3$, sendo 0 o nível mais privilegiado), embora a maioria dos

Tabela 2.2: Tabela de Interrupções do processador Pentium [Patterson and Henessy, 2005]

IRQ	Descrição
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	Intel reserved
16	floating point error
17	alignment check
18	machine check
19-31	Intel reserved
32-255	maskable interrupts (devices & exceptions)

sistemas operacionais construídos para esses processadores só use os níveis extremos (0 para o núcleo do sistema operacional e 3 para utilitários e aplicações)[1].

Na forma mais simples desse esquema, podemos considerar dois níveis básicos de privilégio:

Nível núcleo: também denominado nível *supervisor*, *sistema*, *monitor* ou ainda *kernel space*. Para um código executando nesse nível, todas as funcionalidades do processador estão disponíveis: todas as instruções disponíveis podem ser executadas e todos os recursos internos (registradores e portas de entrada/saída) e áreas de memória podem ser acessados. Ao ser ligado durante a inicialização do computador, o processador entra em operação neste nível.

[1] A ideia de definir níveis de privilégio para o código foi introduzida pelo sistema operacional Multics [Corbató and Vyssotsky, 1965], nos anos 1960. No Multics, os níveis eram organizados em camadas ao redor do núcleo, como em uma cebola, chamadas "anéis de proteção" (*protection rings*).



Figura 2.5: Níveis de privilégio de processadores Intel x86

Nível usuário: neste nível, também chamado *userspace*, somente um subconjunto das instruções do processador e registradores estão disponíveis. Por exemplo, instruções consideradas "perigosas", como `RESET` (reiniciar o processador) e `IN`/`OUT` (acessar portas de entrada/saída), são proibidas. Caso o código em execução tente executar uma instrução proibida, o hardware irá gerar uma exceção, desviando a execução para uma rotina de tratamento dentro do núcleo. Essa rotina irá tratar o erro, provavelmente abortando o programa em execução[2].

Dessa forma, o núcleo do sistema operacional, bem como *drivers* e o código de inicialização, executam em modo núcleo, enquanto os programas utilitários e os aplicativos executam em modo usuário. Os flags que definem o nível de privilégio só podem ser modificados por código executando no nível núcleo, o que impede usuários maliciosos de contornar essa barreira de proteção.

Além da proteção oferecida pelos níveis de privilégio, o núcleo do sistema operacional pode configurar a unidade de gerência de memória (MMU) para criar uma área de memória exclusiva para cada aplicação, isolada das demais aplicações e do núcleo. As aplicações não podem contornar essa barreira de memória, pois a configuração da MMU só pode ser alterada em modo supervisor, ou seja, pelo próprio núcleo.

Com a proteção provida pelos níveis de privilégio do processador e pelas áreas de memória exclusivas configuradas pela MMU, obtém-se um modelo de execução confinada, no qual as aplicações executam isoladas umas das outras e do núcleo, conforme ilustrado na Figura 2.6. Esse modelo de execução é bastante robusto, pois isola os erros, falhas e comportamentos indevidos de uma aplicação do restante do sistema, proporcionando mais estabilidade e segurança.

[2] E também irá gerar a famosa frase "este programa executou uma instrução ilegal e será finalizado", no caso do Windows.

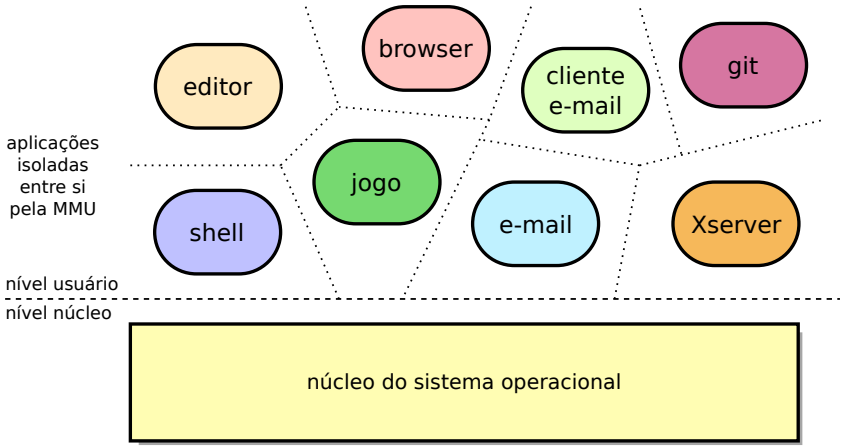


Figura 2.6: Separação entre o núcleo e as aplicações

2.3 Chamadas de sistema

O confinamento de cada aplicação em sua área de memória, imposto pela MMU aos acessos em memória em nível usuário, provê robustez e confiabilidade ao sistema, pois garante que uma aplicação não poderá interferir nas áreas de memória de outras aplicações ou do núcleo. Entretanto, essa proteção introduz um novo problema: como invocar, a partir da aplicação, as rotinas oferecidas pelo núcleo para o acesso ao hardware e demais serviços do SO? Deve-se lembrar que o código do núcleo reside em uma área de memória inacessível à aplicação, então operações de desvio como `jump` e `call` não funcionariam.

A solução desse problema está no mecanismo de interrupção, apresentado na Seção 2.2.2. Quando ocorre uma interrupção, a execução é desviada para um endereço predefinido (desvio incondicional) e o processador passa a operar em nível núcleo (elevação de privilégio). Os processadores atuais implementam uma instrução específica que usa esse mesmo mecanismo para ativar rotinas do núcleo. Ao ser executada, essa instrução comuta o processador para o nível núcleo e executa o código contido em uma rotina predefinida, de forma similar a uma interrupção de hardware. Por essa razão, esse mecanismo é denominado *interrupção de software*, ou *trap*.

A ativação de uma rotina do núcleo usando esse mecanismo é denominada **chamada de sistema** (*system call* ou *syscall*). Os sistemas operacionais definem chamadas de sistema para todas as operações envolvendo o acesso a recursos de baixo nível (periféricos, arquivos, alocação de memória, etc.) ou abstrações lógicas (criação e encerramento de tarefas, operadores de sincronização, etc.). Geralmente as chamadas de sistema são oferecidas para as aplicações em modo usuário através de uma *biblioteca do sistema* (*system library*), que prepara os parâmetros, invoca a chamada e, no retorno desta, devolve à aplicação os resultados obtidos.

Nos processadores modernos a chamada de sistema e seu retorno são feitos usando instruções específicas como `sysenter/sysexit` (x86 32 bits), ou `syscall/sysret` (x86 64 bits). Antes de invocar a chamada de sistema, alguns registradores do processador

são preparados com valores específicos, como o número da operação desejada (*opcode*, geralmente no registrador AX), o endereço dos parâmetros da chamada, etc. As regras para o preenchimento dos registradores são específicas para cada chamada de sistema, em cada sistema operacional.

Um exemplo simples mas ilustrativo do uso de chamadas de sistema em ambiente Linux é apresentado a seguir. O programa em linguagem C imprime uma *string* na saída padrão (chamada `write`) e em seguida encerra a execução (chamada `exit`):

```c
#include <unistd.h>

int main (int argc, char *argv[])
{
  write (1, "Hello World!\n", 13) ; /* write string to stdout */
  _exit (0) ;                      /* exit with no error */
}
```

Esse código em C, ao ser reescrito em Assembly x86, mostra claramente a preparação e invocação das chamadas de sistema:

```
; to assembly and link (in Linux 64 bit):
; nasm -f elf64 -o hello.o hello.asm; ld -o hello hello.o

section .data
msg db 'Hello World!', 0xA  ; output string (0xA = "\n")
len equ 13                  ; string size

section .text

global _start

_start:
   mov rax, 1               ; syscall opcode (1: write)
   mov rdi, 1               ; file descriptor (1: stdout)
   mov rsi, msg             ; data to write
   mov rdx, len             ; number of bytes to write
   syscall                  ; make syscall

   mov rax, 60              ; syscall opcode (60: _exit)
   mov rdi, 0               ; exit status (0: no error)
   syscall                  ; make syscall
```

A Figura 2.7 detalha o funcionamento básico da chamada de sistema `write`, que escreve dados em um arquivo previamente aberto. A seguinte sequência de passos é realizada:

1. No nível usuário, a aplicação invoca a função `write(fd, &buffer, bytes)` da biblioteca de sistema (geralmente a biblioteca padrão da linguagem C).

2. A função `write` preenche os registradores da CPU com os parâmetros recebidos e escreve o *opcode* da chamada de sistema `write` no registrador AX.

3. A função `write` invoca uma chamada de sistema, através da instrução `syscall`.

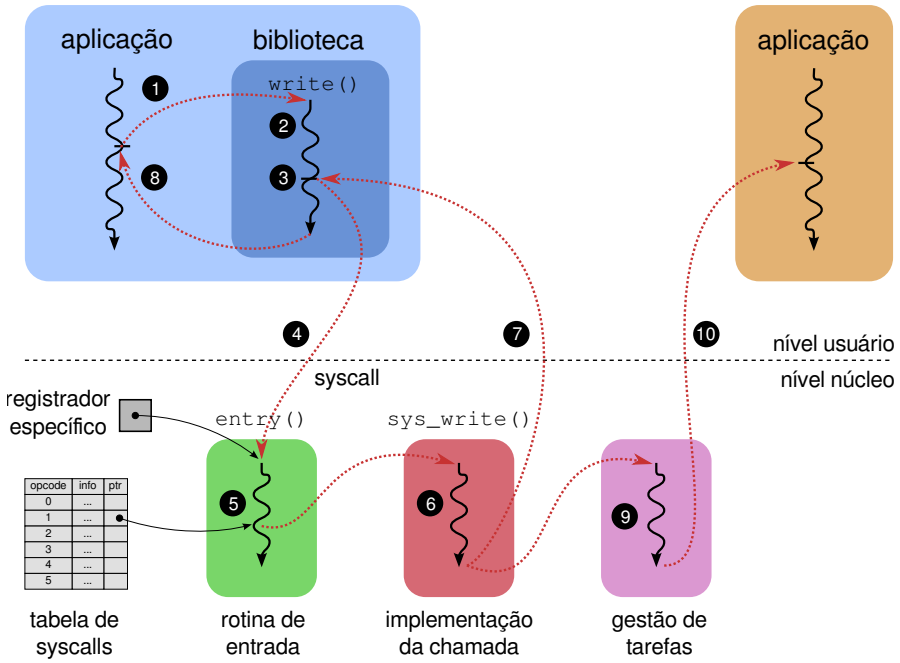


Figura 2.7: Roteiro típico de uma chamada de sistema

4. O processador comuta para o nível privilegiado (*kernel level*) e transfere o controle para a rotina de entrada (`entry`), apontada por um registrador específico.

5. A rotina de entrada recebe em AX o *opcode* da operação desejada (1, `write`), consulta a tabela de chamadas de sistema mantida pelo núcleo e invoca a função que implementa essa operação dentro do núcleo (`sys_write`).

6. A função `sys_write` obtém o endereço dos parâmetros nos demais registradores, verifica a validade dos mesmos e efetua a operação desejada pela aplicação.

7. Ao final da execução da função, eventuais valores de retorno são escritos nos registradores (ou na área de memória da aplicação) e o processamento retorna à função `write`, em modo usuário.

8. A função `write` finaliza sua execução e retorna o controle ao código principal da aplicação.

9. Caso a operação solicitada não possa ser concluida imediatamente, a função `sys_write` passa o controle para a gerência de tarefas, ao invés de retornar para a aplicação solicitante. Isto ocorre, por exemplo, quando é solicitada a leitura de uma entrada do teclado.

10. Na sequência, a gerência de tarefas suspende a execução da aplicação solicitante e devolve o controle do processador a alguma outra aplicação que também esteja

aguardando o retorno de uma chamada de sistema e cuja operação solicitada já tenha sido concluída.

A maioria dos sistemas operacionais implementa centenas de chamadas de sistema distintas, para as mais diversas finalidades. O conjunto de chamadas de sistema oferecidas por um núcleo define a API (*Application Programming Interface*) do sistema operacional. Exemplos de APIs bem conhecidas são a *Win32*, oferecida pelos sistemas Microsoft derivados do Windows NT, e a API *POSIX* [Gallmeister, 1994], que define um padrão de interface de núcleo para sistemas UNIX.

O conjunto de chamadas de sistema de um SO pode ser dividido nas seguintes grandes áreas:

- Gestão de processos: criar, carregar código, terminar, esperar, ler/mudar atributos.
- Gestão da memória: alocar/liberar/modificar áreas de memória.
- Gestão de arquivos: criar, remover, abrir, fechar, ler, escrever, ler/mudar atributos.
- Comunicação: criar/destruir canais de comunicação, receber/enviar dados.
- Gestão de dispositivos: ler/mudar configurações, ler/escrever dados.
- Gestão do sistema: ler/mudar data e hora, desligar/suspender/reiniciar o sistema.

Exercícios

1. O que diferencia o *núcleo* do restante do sistema operacional?
2. Seria possível construir um sistema operacional seguro usando um processador que não tenha níveis de privilégio? Por quê?
3. Os processadores da família x86 possuem dois bits para definir o nível de privilégio, resultando em 4 níveis distintos. A maioria dos sistemas operacionais para esses processadores usam somente os níveis extremos (0 e 3, ou 00_2 e 11_2). Haveria alguma utilidade para os níveis intermediários?
4. Quais as diferenças entre *interrupções*, *exceções* e *traps*?
5. O comando em linguagem C `fopen` é uma chamada de sistema ou uma função de biblioteca? Por quê?
6. A operação em modo usuário permite ao processador executar somente parte das instruções disponíveis em seu conjunto de instruções. Quais das seguintes operações não deveriam ser permitidas em nível usuário? Por quê?
   (a) Ler uma porta de entrada/saída
   (b) Efetuar uma divisão inteira
   (c) Escrever um valor em uma posição de memória
   (d) Ajustar o valor do relógio do hardware

(e) Ler o valor dos registradores do processador

(f) Mascarar uma ou mais interrupções

7. Considerando um processo em um sistema operacional com proteção de memória entre o núcleo e as aplicações, indique quais das seguintes ações do processo teriam de ser realizadas através de chamadas de sistema, justificando suas respostas:

   (a) Ler o relógio de tempo real do hardware.

   (b) Enviar um pacote através da rede.

   (c) Calcular uma exponenciação.

   (d) Preencher uma área de memória do processo com zeros.

   (e) Remover um arquivo do disco.

8. Coloque na ordem correta as ações abaixo, que ocorrem durante a execução da função `printf("Hello world")` por um processo (observe que nem todas as ações indicadas fazem parte da sequência).

   (a) A rotina de tratamento da interrupção de software é ativada dentro do núcleo.

   (b) A função `printf` finaliza sua execução e devolve o controle ao código do processo.

   (c) A função de biblioteca `printf` recebe e processa os parâmetros de entrada (a string "`Hello world`").

   (d) A função de biblioteca `printf` prepara os registradores para solicitar a chamada de sistema `write()`

   (e) O disco rígido gera uma interrupção indicando a conclusão da operação.

   (f) O escalonador escolhe o processo mais prioritário para execução.

   (g) Uma interrupção de software é acionada.

   (h) O processo chama a função `printf` da biblioteca C.

   (i) A operação de escrita no terminal é efetuada ou agendada pela rotina de tratamento da interrupção.

   (j) O controle volta para a função `printf` em modo usuário.

Atividades

1. O utilitário `strace` do UNIX permite observar a sequência de **chamadas de sistema** efetuadas por uma aplicação. Em um terminal, execute `strace date` para descobrir quais os arquivos abertos pela execução do utilitário `date` (que indica a data e hora correntes). Por que o utilitário `date` precisa fazer chamadas de sistema?

2. O utilitário `ltrace` do UNIX permite observar a sequência de **chamadas de biblioteca** efetuadas por uma aplicação. Em um terminal, execute `ltrace date` para descobrir as funções de biblioteca chamadas pela execução do utilitário `date` (que indica a data e hora correntes). Pode ser observada alguma relação entre as chamadas de biblioteca e as chamadas de sistema observadas no item anterior?

Referências

F. J. Corbató and V. A. Vyssotsky. Introduction and overview of the Multics system. In *AFIPS Conference Proceedings*, pages 185–196, 1965.

B. Gallmeister. *POSIX.4: Programming for the Real World*. O'Reilly Media, Inc, 1994.

Google. *Android Platform Architecture*, April 2018. https://developer.android.com/guide/platform.

D. Patterson and J. Henessy. *Organização e Projeto de Computadores*. Campus, 2005.

W. Stallings. *Arquitetura e organização de computadores. 8ª ed*. Pearson, 2010.

Capítulo 3

Arquiteturas de SOs

Embora a definição de níveis de privilégio (Seção 2.3) imponha uma estruturação mínima a um sistema operacional, forçando a separação entre o núcleo e o espaço de usuário, os vários elementos que compõem o sistema podem ser organizados de diversas formas, separando suas funcionalidades e modularizando seu projeto. Neste capítulo serão apresentadas as arquiteturas mais populares para a organização de sistemas operacionais.

3.1 Sistemas monolíticos

Em um sistema monolítico[1], o sistema operacional é um "bloco maciço" de código que opera em modo núcleo, com acesso a todos os recursos do hardware e sem restrições de acesso à memória. Por isso, os componentes internos do sistema operacional podem se relacionar entre si conforme suas necessidades. A Figura 3.1 ilustra essa arquitetura.

A grande vantagem da arquitetura monolítica é seu desempenho: qualquer componente do núcleo pode acessar os demais componentes, áreas de memória ou mesmo dispositivos periféricos diretamente, pois não há barreiras impedindo esses acessos. A interação direta entre componentes leva a sistemas mais rápidos e compactos, pois não há necessidade de utilizar mecanismos específicos de comunicação entre os componentes do núcleo.

Todavia, a arquitetura monolítica pode levar a problemas de robustez do sistema. Como todos os componentes do SO têm acesso privilegiado ao hardware, caso um componente perca o controle devido a algum erro (acessando um ponteiro inválido ou uma posição inexistente em um vetor, por exemplo), esse erro pode se propagar rapidamente por todo o núcleo, levando o sistema ao colapso (travamento, reinicialização ou funcionamento errático).

Outro problema relacionado às arquiteturas monolíticas diz respeito ao processo de desenvolvimento. Como os componentes do sistema têm acesso direto uns aos outros, podem ser fortemente interdependentes, tornando a manutenção e evolução do núcleo mais complexas. Como as dependências e pontos de interação entre os componentes podem ser pouco evidentes, pequenas alterações na estrutura de dados de um componente

[1] A palavra "monólito" vem do grego *monos* (único ou unitário) e *lithos* (pedra).

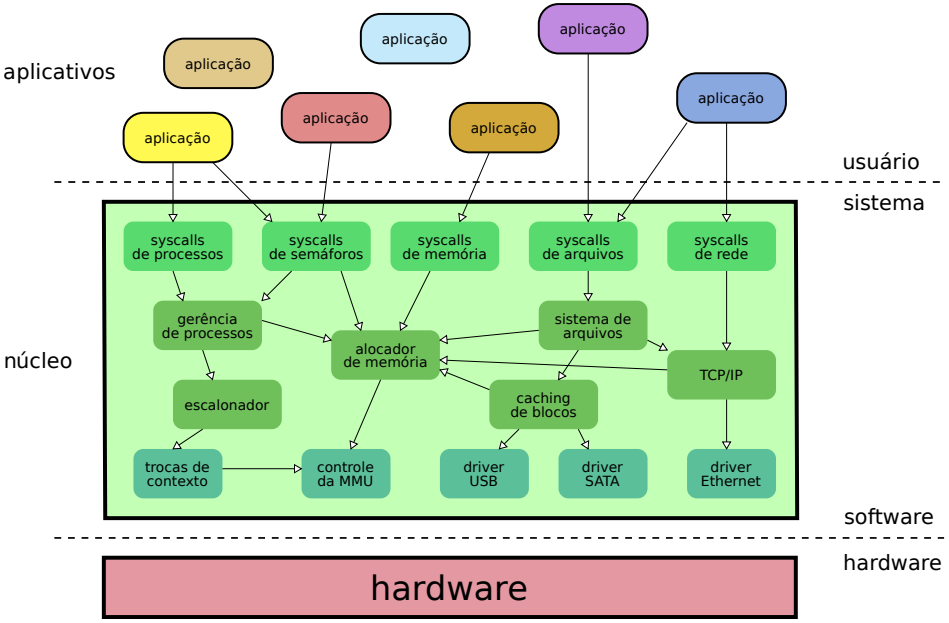


Figura 3.1: Núcleo de sistema operacional monolítico.

podem ter um impacto inesperado em outros componentes, caso estes acessem aquela estrutura diretamente.

A arquitetura monolítica foi a primeira forma de organizar os sistemas operacionais; sistemas UNIX antigos e o MS-DOS seguiam esse modelo. O núcleo do Linux é monolítico, mas seu código vem sendo gradativamente estruturado e modularizado desde a versão 2.0 (embora boa parte do código ainda permaneça no nível de núcleo). O sistema FreeBSD [McKusick and Neville-Neil, 2005] também usa um núcleo monolítico.

3.2 Sistemas micronúcleo

Outra possibilidade de estruturar o SO consiste em retirar do núcleo todo o código de alto nível, normalmente associado às abstrações de recursos, deixando no núcleo somente o código de baixo nível necessário para interagir com o hardware e criar algumas abstrações básicas. O restante do código seria transferido para programas separados no espaço de usuário, denominados *serviços*. Por fazer o núcleo de sistema ficar menor, essa abordagem foi denominada *micronúcleo* (ou *µ-kernel*).

A abordagem micronúcleo oferece maior modularidade, pois cada serviço pode ser desenvolvido de forma independente dos demais; mais flexibilidade, pois os serviços podem ser carregados e desativados conforme a necessidade; e mais robustez, pois caso um serviço falhe, somente ele será afetado, devido ao confinamento de memória entre os serviços.

Um micronúcleo normalmente implementa somente a noção de tarefa, os espaços de memória protegidos para cada aplicação, a comunicação entre tarefas e as operações de acesso às portas de entrada/saída (para acessar os dispositivos). Todos os aspectos de alto nível, como políticas de uso do processador e da memória, o sistema de arquivos, o controle de acesso aos recursos e até mesmos os *drivers* são implementados fora do núcleo, em processos que se comunicam usando o mecanismo de comunicação provido pelo núcleo.

Um bom exemplo de sistema micronúcleo é o Minix 3 [Herder et al., 2006]. Neste sistema, o núcleo oferece funcionalidades básicas de gestão de interrupções, configuração da CPU e da MMU, acesso às portas de entrada/saída e primitivas de troca de mensagens entre aplicações. Todas as demais funcionalidades, como a gestão de arquivos e de memória, protocolos de rede, políticas de escalonamento, etc., são providas por processos fora do núcleo, denominados *servidores*. A Figura 3.2 apresenta a arquitetura do Minix 3:

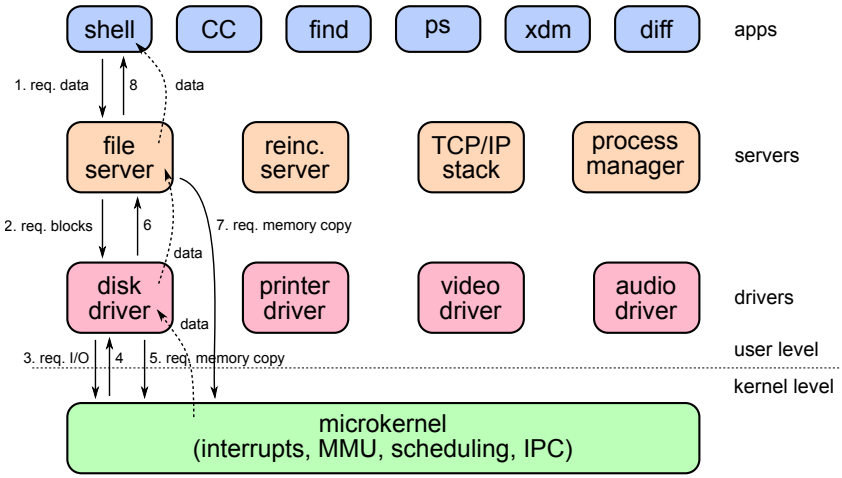


Figura 3.2: Visão geral da arquitetura do MINIX 3 (adaptado de [Herder et al., 2006]).

Por exemplo, no sistema Minix 3 a seguinte sequência de ações ocorre quando uma aplicação deseja ler dados de um arquivo no disco:

1. usando as primitivas de mensagens do núcleo, a aplicação envia uma mensagem (m_1) ao servidor de arquivos, solicitando a leitura de dados de um arquivo;
2. o servidor de arquivos verifica se possui os dados em seu cache local; se não os possuir, envia uma mensagem (m_2) ao *driver* de disco solicitando a leitura dos dados do disco em seu espaço de memória;
3. o *driver* de disco envia uma mensagem (m_3) ao núcleo solicitando operações nas portas de entrada/saída do controlador de disco, para ler dados do mesmo;
4. o núcleo verifica se o *driver* de disco tem permissão para usar as portas de entrada/saída e agenda a operação solicitada;
5. quando o disco concluir a operação, o núcleo transfere os dados lidos para a memória do *driver* de disco e responde (m_4) a este;

6. o *driver* de disco solicita (m_5) então ao núcleo a cópia dos dados recebidos para a memória do servidor de arquivos e responde (m_6) à mensagem anterior deste, informando a conclusão da operação;

7. o servidor de arquivos solicita (m_7) ao núcleo a cópia dos dados recebidos para a memória da aplicação e responde (m_8) à mensagem anterior desta;

8. a aplicação recebe a resposta de sua solicitação e usa os dados lidos.

Da sequência acima pode-se perceber que foram necessárias 8 mensagens (m_i) para realizar uma leitura de dados do disco, cada uma correspondendo a uma chamada de sistema. Cada chamada de sistema tem um custo elevado, pois implica na mudança do fluxo de execução e reconfiguração da memória acessível pela MMU. Além disso, foram necessárias várias cópias de dados entre as áreas de memória de cada entidade. Esses fatores levam a um desempenho bem inferior ao da abordagem monolítica, razão que dificulta a adoção plena dessa abordagem.

O micronúcleos vem sendo investigados desde os anos 1980. Dois exemplos clássicos dessa abordagem são os sistemas Mach [Rashid et al., 1989] e Chorus [Rozier and Martins, 1987]. Os melhores exemplos de micronúcleos bem sucedidos são provavelmente o Minix 3 [Herder et al., 2006] e o QNX. Contudo, vários sistemas operacionais atuais adotam parcialmente essa estruturação, adotando núcleos híbridos, como por exemplo o MacOS X da Apple, o Digital UNIX e o Windows NT.

3.3 Sistemas em camadas

Uma forma mais elegante de estruturar um sistema operacional faz uso da noção de camadas: a camada mais baixa realiza a interface com o hardware, enquanto as camadas intermediárias proveem níveis de abstração e gerência cada vez mais sofisticados. Por fim, a camada superior define a interface do núcleo para as aplicações (as chamadas de sistema). As camadas têm níveis de privilégio decrescentes: a camada inferior tem acesso total ao hardware, enquanto a superior tem acesso bem mais restrito (vide Seção 2.2.3).

A abordagem de estruturação de software em camadas teve sucesso no domínio das redes de computadores, através do modelo de referência OSI (*Open Systems Interconnection*) [Day, 1983], e também seria de se esperar sua adoção no domínio dos sistemas operacionais. No entanto, alguns inconvenientes limitam a aplicação do modelo em camadas de forma intensiva nos sistemas operacionais:

- O empilhamento de várias camadas de software faz com que cada pedido de uma aplicação demore mais tempo para chegar até o dispositivo periférico ou recurso a ser acessado, prejudicando o desempenho.

- Nem sempre a divisão de funcionalidades do sistema em camadas é óbvia, pois muitas dessas funcionalidades são interdependentes e não teriam como ser organizadas em camadas. Por exemplo, a gestão de entrada/saída necessita de serviços de memória para alocar/liberar *buffers* para leitura e escrita de dados, mas a gestão de memória precisa da gestão de entrada/saída para implementar a paginação em disco (*paging*). Qual dessas duas camadas viria antes?

Em decorrência desses inconvenientes, a estruturação em camadas é apenas parcialmente adotada hoje em dia. Como exemplo de sistema fortemente estruturado em camadas pode ser citado o MULTICS [Corbató and Vyssotsky, 1965].

Muitos sistemas mais recentes implementam uma camada inferior de abstração do hardware para interagir com os dispositivos (a camada *HAL – Hardware Abstraction Layer*, implementada no Windows NT e seus sucessores), e também organizam em camadas alguns subsistemas, como a gerência de arquivos e o suporte de rede (seguindo o modelo OSI). Essa organização parcial em camadas pode ser facilmente observada nas arquiteturas do Minix 3 (Figura 3.2) Windows 2000 (Figura 3.3) e Android (Figura 2.2).

3.4 Sistemas híbridos

Apesar das boas propriedades de modularidade, flexibilidade e robustez proporcionadas pelos micronúcleos, sua adoção não teve o sucesso esperado devido ao baixo desempenho. Uma solução encontrada para esse problema consiste em trazer de volta ao núcleo os componentes mais críticos, para obter melhor desempenho. Essa abordagem intermediária entre o núcleo monolítico e micronúcleo é denominada *núcleo híbrido*. É comum observar também nos núcleos híbridos uma influência da arquitetura em camadas.

Vários sistemas operacionais atuais empregam núcleos híbridos, entre eles o Microsoft Windows, a partir do Windows NT. As primeiras versões do Windows NT podiam ser consideradas micronúcleo, mas a partir da versão 4 vários subsistemas foram movidos para dentro do núcleo para melhorar seu desempenho, transformando-o em um núcleo híbrido. Os sistemas MacOS e iOS da Apple também adotam um núcleo híbrido chamado XNU (de *X is Not Unix*), construído a partir dos núcleos Mach (micronúcleo) [Rashid et al., 1989] e FreeBSD (monolítico) [McKusick and Neville-Neil, 2005]. A figura 3.3 mostra a arquitetura interna do núcleo usado no SO Windows 2000.

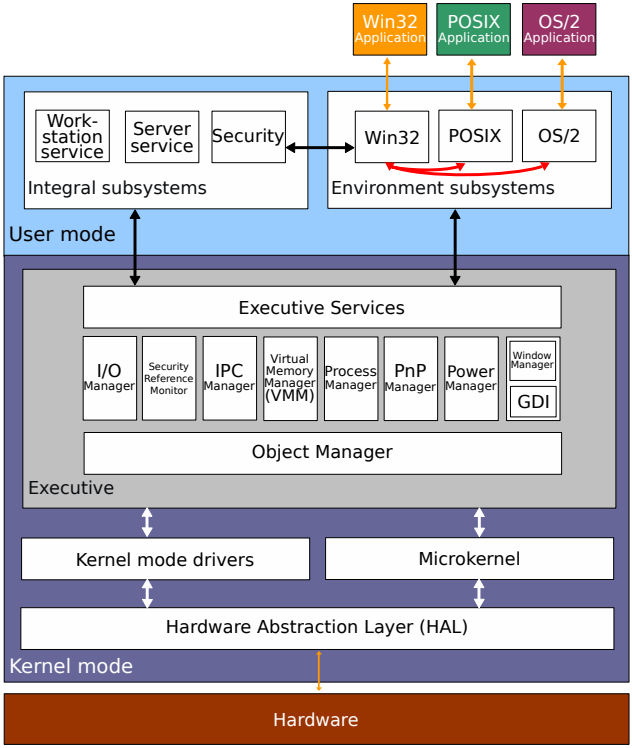


Figura 3.3: Arquitetura do sistema Windows 2000 [Wikipedia, 2018].

3.5 Arquiteturas avançadas

Além das arquiteturas clássicas (monolítica, em camadas, micronúcleo), recentemente surgiram várias propostas organizar os componentes do sistema operacional, com vistas a contextos de aplicação específicos, como nuvens computacionais. Esta seção apresenta algumas dessas novas arquiteturas.

3.5.1 Máquinas virtuais

Apesar de ser um conceito dos anos 1960 [Goldberg, 1973], a virtualização de sistemas e de aplicações ganhou um forte impulso a partir dos anos 2000, com a linguagem Java, a consolidação de servidores (juntar vários servidores com seus sistemas operacionais em um único computador físico) e, mais recentemente, a computação em nuvem.

Uma máquina virtual é uma camada de software que "transforma" um sistema em outro, ou seja, que usa os serviços fornecidos por um sistema operacional (ou pelo hardware) para construir a interface de outro sistema. Por exemplo, o ambiente (JVM - *Java Virtual Machine*) usa os serviços de um sistema operacional convencional (Windows ou Linux) para construir um computador virtual que processa *bytecode*, o código binário

dos programas Java. Da mesma forma, o ambiente *VMWare* permite executar o sistema Windows sobre um sistema Linux subjacente, ou vice-versa.

Um ambiente de máquinas virtuais consiste de três partes básicas, que podem ser observadas nos exemplos da Figura 3.4:

- O sistema real, ou sistema hospedeiro (*host*), que contém os recursos reais de hardware e software do sistema;
- a camada de virtualização, denominada *hipervisor* ou *monitor de virtualização* (*VMM - Virtual Machine Monitor*), que constrói a máquina virtual a partir dos recursos do sistema real;
- o sistema virtual, ou sistema convidado (*guest*); em alguns casos, vários sistemas virtuais podem coexistir, executando sobre o mesmo sistema real.

Existem diversas possibilidades de uso da virtualização, levando a vários tipos de hipervisor, por exemplo:

Hipervisor de aplicação: suporta a execução de uma única aplicação em uma linguagem específica. As máquinas virtuais JVM (*Java Virtual Machine*) e CLR (*Common Language Runtime*, para C#) são exemplos típicos dessa abordagem.

Hipervisor de sistema: suporta a execução de um sistema operacional convidado com suas aplicações, como nos dois exemplos da Figura 3.4. Esta categoria se subdivide em:

Hipervisor nativo: executa diretamente sobre o hardware, virtualizando os recursos deste para construir máquinas virtuais, onde executam vários sistemas operacionais com suas respectivas aplicações. O exemplo da esquerda na Figura 3.4 ilustra esse conceito. Um exemplo deste tipo de hipervisor é o ambiente *VMware ESXi Server* [Newman et al., 2005].

Hipervisor convidado: executa sobre um sistema operacional hospedeiro, como no exemplo da direita na Figura 3.4. Os hipervisores *VMWare Workstation* e *VirtualBox* implementam essa abordagem.

Por permitir a execução de vários sistemas independentes e isolados entre si sobre o mesmo hardware, hipervisores de sistema são muito usados em ambientes corporativos de larga escala, como as nuvens computacionais. Hipervisores de aplicação também são muito usados, por permitir a execução de código independente de plataforma, como em Java e C#. Avanços recentes no suporte de hardware para a virtualização e na estrutura interna dos hipervisores permitem a execução de máquinas virtuais com baixo custo adicional em relação à execução nativa.

O capítulo 31.4 deste livro contém uma apresentação mais detalhada do conceito de virtualização e de suas possibilidades de uso.

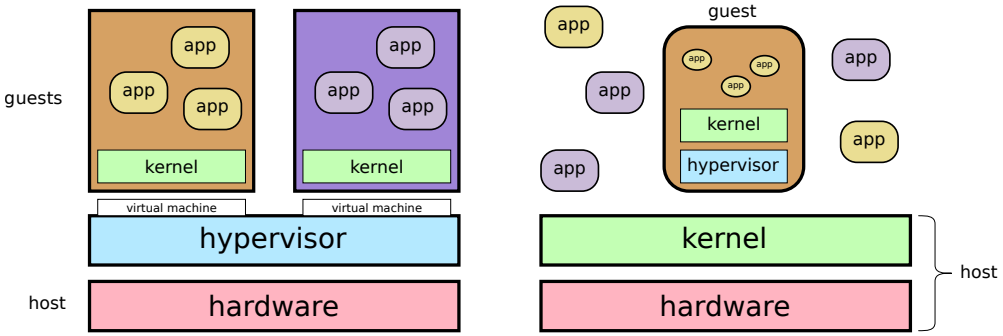


Figura 3.4: Ambientes de virtualização nativo (esq.) e convidado (dir.).

3.5.2 Contêineres

Além da virtualização, outra forma de isolar aplicações ou subsistemas em um sistema operacional consiste na virtualização do espaço de usuário (*userspace*). Nesta abordagem, denominada *servidores virtuais* ou *contêineres*, o espaço de usuário do sistema operacional é dividido em áreas isoladas denominadas *domínios* ou *contêineres*. A cada domínio é alocada uma parcela dos recursos do sistema operacional, como memória, tempo de processador e espaço em disco.

Para garantir o isolamento entre os domínios, cada domínio tem sua própria interface de rede virtual e, portanto, seu próprio endereço de rede. Além disso, na maioria das implementações cada domínio tem seu próprio espaço de nomes para os identificadores de usuários, processos e primitivas de comunicação. Assim, é possível encontrar um usuário `pedro` no domínio d_3 e outro usuário `pedro` no domínio d_7, sem relação entre eles nem conflitos. Essa noção de espaços de nomes distintos se estende aos demais recursos do sistema: identificadores de processos, semáforos, árvores de diretórios, etc. Entretanto, o núcleo do sistema operacional é o mesmo para todos os domínios.

Processos em um mesmo domínio podem interagir entre si normalmente, mas processos em domínios distintos não podem ver ou interagir entre si. Além disso, processos não podem trocar de domínio nem acessar recursos de outros domínios. Dessa forma, os domínios são vistos como sistemas distintos, acessíveis somente através de seus endereços de rede. Para fins de gerência, normalmente é definido um domínio d_0, chamado de *domínio inicial*, *privilegiado* ou *de gerência*, cujos processos têm visibilidade e acesso aos processos e recursos dos demais domínios.

A Figura 3.5 mostra a estrutura típica de um ambiente de contêineres. Nela, pode-se observar que um processo pode migrar de d_0 para d_1, mas que os processos em d_1 não podem migrar para outros domínios. A comunicação entre processos confinados em domínios distintos (d_2 e d_3) também é proibida.

Existem implementações de contêineres, como as *Jails* do sistema FreeBSD [McKusick and Neville-Neil, 2005] ou as *zonas* do sistema Solaris [Price and Tucker, 2004], que oferecem o isolamento entre domínios e o controle de uso dos recursos pelos mesmos. O núcleo Linux oferece diversos mecanismos para o isolamento de espaços de recursos,

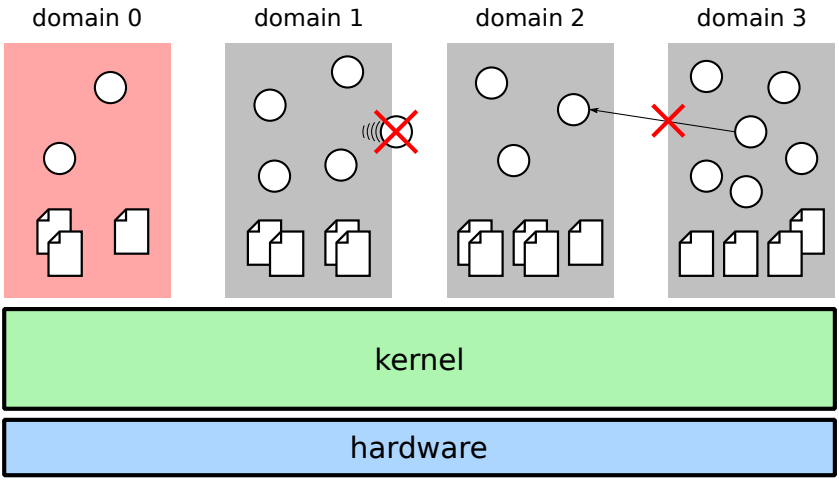


Figura 3.5: Sistema com contêineres (domínios).

que são usados por plataformas de gerenciamento de contêineres como *Linux Containers* (LXC), *Docker* e *Kubernetes*.

3.5.3 Sistemas exonúcleo

Em um sistema convencional, as aplicações executam sobre uma pilha de serviços de abstração e gerência de recursos providos pelo sistema operacional. Esses serviços facilitam a construção de aplicações, mas podem constituir uma sobrecarga significativa para o desempenho do sistema. Para cada abstração ou serviço utilizado é necessário interagir com o núcleo através de chamadas de sistema, que impactam negativamente o desempenho. Além disso, os serviços providos pelo núcleo devem ser genéricos o suficiente para atender a uma vasta variedade de aplicações. Aplicações com demandas muito específicas, como um servidor Web de alto desempenho, nem sempre são atendidas da melhor forma possível.

Os exonúcleos (*exokernels*) [Engler, 1998] tentam trazer uma resposta a essas questões, reduzindo ainda mais os serviços oferecidos pelo núcleo. Em um sistema exonúcleo, o núcleo do sistema apenas proporciona acesso controlado aos recursos do hardware, mas não implementa nenhuma abstração. Por exemplo, o núcleo provê acesso compartilhado à interface de rede, mas não implementa nenhum protocolo. Todas as abstrações e funcionalidades de gerência que uma aplicação precisa terão de ser implementadas pela própria aplicação, em seu espaço de memória.

Para simplificar a construção de aplicações sobre exonúcleos, são usados sistemas operacionais "de biblioteca" ou *LibOS* (*Library Operating System*), que implementam as abstrações usuais, como memória virtual, sistemas de arquivos, semáforos e protocolos de rede. Um LibOS nada mais é que um conjunto de bibliotecas compartilhadas usadas pela aplicação para prover os serviços de que esta necessita. Diferentes aplicações sobre o mesmo exonúcleo podem usar LibOS diferentes, ou implementar suas próprias abstrações. A Figura 3.6 ilustra a arquitetura de um exonúcleo típico.

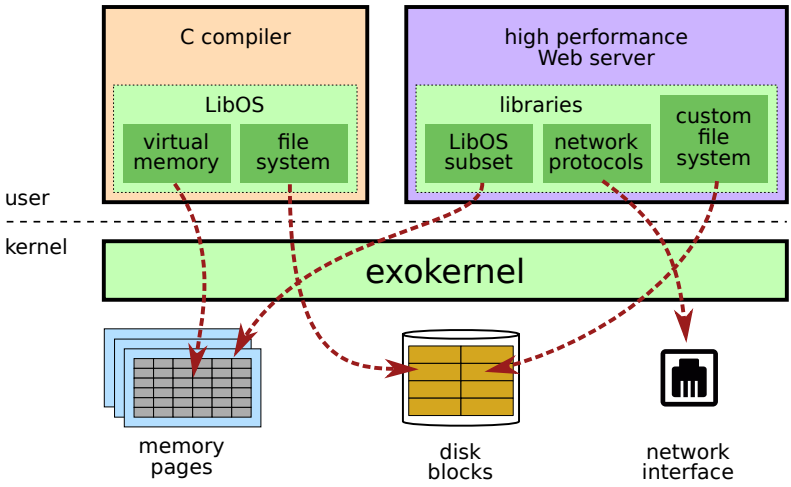


Figura 3.6: Sistema exonúcleo (adaptado de [Engler, 1998]).

É importante observar que um exonúcleo é diferente de um micronúcleo, pois neste último as abstrações são implementada por um conjunto de processos servidores independentes e isolados entre si, enquanto no exonúcleo cada aplicação implementa suas próprias abstrações (ou as incorpora de bibliotecas compartilhadas).

Até o momento não existem exonúcleos em produtos comerciais, as implementações mais conhecidas são esforços de projetos de pesquisa, como Aegis/ExOS [Engler, 1998] e Nemesis [Hand, 1999].

3.5.4 Sistemas uninúcleo

Nos sistemas uninúcleo (*unikernel*) [Madhavapeddy and Scott, 2013], um núcleo de sistema operacional, as bibliotecas e uma aplicação são compilados e ligados entre si, formando um bloco monolítico de código, que executa em um único espaço de endereçamento, em modo privilegiado. Dessa forma, o custo da transição aplicação/núcleo nas chamadas de sistema diminui muito, gerando um forte ganho de desempenho.

Outro ponto positivo da abordagem uninúcleo é o fato de incluir no código final somente os componentes necessários para suportar a aplicação-alvo e os *drivers* necessários para acessar o hardware-alvo, levando a um sistema compacto, que pode ser lançado rapidamente.

A estrutura de uninúcleo se mostra adequada para computadores que executam uma única aplicação, como servidores de rede (HTTP, DNS, DHCP) e serviços em ambientes de nuvem computacional (bancos de dados, Web Services). De fato, o maior interesse em usar uninúcleos vem do ambiente de nuvem, pois os "computadores" da nuvem são na verdade máquinas virtuais com hardware virtual simples (com *drivers* padronizados) e que executam serviços isolados (vide Seção 3.5.1).

Os sistemas OSv [Kivity et al., 2014] e MirageOS [Madhavapeddy and Scott, 2013] são bons exemplos de uninúcleos. Como eles foram especialmente concebidos para

as nuvens, são também conhecidos como "CloudOS". A Figura 3.7 mostra a arquitetura desses sistemas.

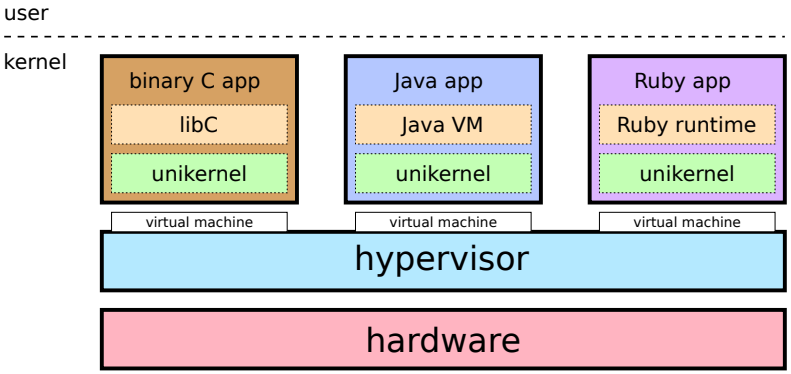


Figura 3.7: Sistema uninúcleo.

É interessante observar que, em sistemas embarcados e nas chamadas *appliances* de rede (roteadores, modems, etc), é usual compilar o núcleo, bibliotecas e aplicação em um único arquivo binário que é em seguida executado diretamente sobre o hardware, em modo privilegiado. Este é o caso, por exemplo, do sistema TinyOS [Levis et al., 2005], voltado para aplicações de Internet das Coisas (IoT - *Internet of Things*). Apesar desse sistema não ser oficialmente denominado um uninúcleo, a abordagem basicamente é a mesma.

Exercícios

1. Monte uma tabela com os benefícios e deficiências mais relevantes das principais arquiteturas de sistemas operacionais.
2. O Linux possui um núcleo similar com o da figura 3.1, mas também possui "tarefas de núcleo" que executam como os gerentes da figura 3.2. Seu núcleo é monolítico ou micronúcleo? Por quê?
3. Sobre as afirmações a seguir, relativas às diversas arquiteturas de sistemas operacionais, indique quais são incorretas, justificando sua resposta:
   (a) Uma máquina virtual de sistema é construída para suportar uma aplicação escrita em uma linguagem de programação específica, como Java.
   (b) Um hipervisor convidado executa sobre um sistema operacional hospedeiro.
   (c) Em um sistema operacional micronúcleo, os diversos componentes do sistema são construídos como módulos interconectados executando dentro do núcleo.
   (d) Núcleos monolíticos são muito utilizados devido à sua robustez e facilidade de manutenção.
   (e) Em um sistema operacional micronúcleo, as chamadas de sistema são implementadas através de trocas de mensagens.

Atividades

1. O sistema operacional Minix 3 usa uma arquitetura micronúcleo. Ele pode ser obtido gratuitamente no site `http://www.minix3.org`. Instale-o em uma máquina virtual e explore seus processos, usando os comandos `top` e `ps`. Identifique os principais processos em execução, usando a documentação do site.

Referências

F. J. Corbató and V. A. Vyssotsky. Introduction and overview of the Multics system. In *AFIPS Conference Proceedings*, pages 185–196, 1965.

J. Day. The OSI reference model. *Proceedings of the IEEE*, December 1983.

D. Engler. *The Exokernel Operating System Architecture*. PhD thesis, Massachusetts Institute of Technology, Cambridge - MA, October 1998.

R. Goldberg. Architecture of virtual machines. In *AFIPS National Computer Conference*, 1973.

S. Hand. Self-paging in the Nemesis operating system. In *3rd Symposium on Operating Systems Design and Implementation (OSDI)*, volume 99, pages 73–86, 1999.

J. N. Herder, H. Bos, B. Gras, P. Homburg, and A. S. Tanenbaum. Minix 3: A highly reliable, self-repairing operating system. *ACM SIGOPS Operating Systems Review*, 40(3), Sept. 2006.

A. Kivity, D. L. G. Costa, and P. Enberg. OSv – optimizing the operating system for virtual machines. In *USENIX Annual Technical Conference*, page 61, 2014.

P. Levis, S. Madden, J. Polastre, R. Szewczyk, K. Whitehouse, A. Woo, D. Gay, J. Hill, M. Welsh, E. Brewer, et al. TinyOS: An operating system for sensor networks. In *Ambient intelligence*, pages 115–148. Springer, 2005.

A. Madhavapeddy and D. J. Scott. Unikernels: Rise of the virtual library operating system. *Queue*, 11(11):30:30–30:44, Dec. 2013. ISSN 1542-7730.

M. McKusick and G. Neville-Neil. *The Design and Implementation of the FreeBSD Operating System*. Pearson Education, 2005.

M. Newman, C.-M. Wiberg, and B. Braswell. *Server Consolidation with VMware ESX Server*. IBM RedBooks, 2005. `http://www.redbooks.ibm.com`.

D. Price and A. Tucker. Solaris zones: Operating system support for consolidating commercial workloads. In *18th USENIX conference on System administration*, pages 241–254, 2004.

R. Rashid, D. Julin, D. Orr, R. Sanzi, R. Baron, A. Forin, D. Golub, and M. B. Jones. Mach: a system software kernel. In *Proceedings of the 1989 IEEE International Conference, COMPCON*, pages 176–178, San Francisco, CA, USA, 1989. IEEE Comput. Soc. Press.

M. Rozier and J. L. Martins. The Chorus distributed operating system: Some design issues. In Y. Paker, J.-P. Banatre, and M. Bozyiğit, editors, *Distributed Operating Systems*, pages 261–287, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.

Wikipedia. Wikipedia online enciclopedia. http://www.wikipedia.org, 2018.

Parte II

Gestão de tarefas

Capítulo 4

O conceito de tarefa

Um sistema de computação quase sempre tem mais atividades a executar que o número de processadores disponíveis. Assim, é necessário criar métodos para multiplexar o(s) processador(es) da máquina entre as tarefas presentes. Além disso, como as diferentes tarefas têm necessidades distintas de processamento e nem sempre a capacidade de processamento existente é suficiente para atender a todos, estratégias precisam ser definidas para que cada tarefa receba uma quantidade de processamento que atenda suas necessidades. Este capítulo apresenta os principais conceitos, estratégias e mecanismos empregados na gestão do processador e das tarefas em execução em um sistema de computação.

4.1 Objetivos

Em um sistema de computação, é frequente a necessidade de executar várias tarefas distintas simultaneamente. Por exemplo:

- O usuário de um computador pessoal pode estar editando uma imagem, imprimindo um relatório, ouvindo música e trazendo da Internet um novo software, tudo ao mesmo tempo.
- Em um grande servidor de e-mails, milhares de usuários conectados remotamente enviam e recebem e-mails através da rede.
- Um navegador Web precisa buscar os elementos da página a exibir, analisar e renderizar o código HTML e os gráficos recebidos, animar os elementos da interface e responder aos comandos do usuário.

Por outro lado, um processador convencional somente trata um fluxo de instruções de cada vez. Até mesmo computadores com vários processadores, vários *cores* ou com tecnologia *hyper-threading*, por exemplo, têm mais tarefas a executar que o número de processadores disponíveis. Como fazer para atender simultaneamente as múltiplas necessidades de processamento dos usuários?

Uma solução ingênua para esse problema seria equipar o sistema com um processador para cada tarefa, mas essa solução ainda é inviável econômica e tecnicamente. Outra

solução seria *multiplexar o processador* entre as várias tarefas que requerem processamento, ou seja, compartilhar o uso do processador entre as várias tarefas, de forma a atendê-las da melhor maneira possível.

Para uma gestão eficaz do processamento, é fundamental compreender o conceito de *tarefa*. O restante deste capítulo aborda o conceito de tarefa, como estas são definidas, quais os seus estados possíveis e como/quando elas mudam de estado.

4.2 O conceito de tarefa

Uma tarefa é definida como sendo a execução de um fluxo sequencial de instruções, construído para atender uma finalidade específica: realizar um cálculo complexo, a edição de um gráfico, a formatação de um disco, etc. Assim, a execução de uma sequência de instruções em linguagem de máquina, normalmente gerada pela compilação de um programa escrito em uma linguagem qualquer, é denominada "tarefa" ou "atividade" (do inglês *task*).

É importante ressaltar as diferenças entre os conceitos de *tarefa* e de *programa*:

Programa: é um conjunto de uma ou mais sequências de instruções escritas para resolver um problema específico, constituindo assim uma aplicação ou utilitário. O programa representa um conceito *estático*, sem um estado interno definido (que represente uma situação específica da execução) e sem interações com outras entidades (o usuário ou outros programas). Os arquivos `C:\Windows\notepad.exe` e `/usr/bin/vi` são exemplos de programas (no caso, para edição de texto).

Tarefa: é a execução sequencial, por um processador, da sequência de instruções definidas em um programa para realizar seu objetivo. Trata-se de um conceito *dinâmico*, que possui um estado interno bem definido a cada instante (os valores das variáveis internas e a posição atual da execução evoluem com o tempo) e interage com outras entidades: o usuário, os dispositivos periféricos e/ou outras tarefas. Tarefas podem ser implementadas de várias formas, como processos (Seção 5.3) ou *threads* (Seção 5.4).

Fazendo uma analogia simples, pode-se dizer que um programa é o equivalente de uma "receita de torta" dentro de um livro de receitas (um diretório) guardado em uma estante (um disco) na cozinha (o computador). Essa receita de torta define os ingredientes necessários (entradas) e o modo de preparo (programa) da torta (saída). Por sua vez, a ação de "executar" a receita, providenciando os ingredientes e seguindo os passos definidos na mesma, é a tarefa propriamente dita. A cada momento, a cozinheira (o processador) está seguindo um passo da receita (posição da execução) e tem uma certa disposição dos ingredientes e utensílios em uso (as entradas e variáveis internas da tarefa).

Assim como uma receita de torta pode definir várias atividades interdependentes para elaborar a torta (preparar a massa, fazer o recheio, assar, decorar, etc.), um programa também pode definir várias sequências de execução interdependentes para atingir seus objetivos. Por exemplo, o programa do navegador Web ilustrado na Figura 4.1 define várias tarefas que uma janela de navegador deve executar simultaneamente, para que o usuário possa navegar na Internet:

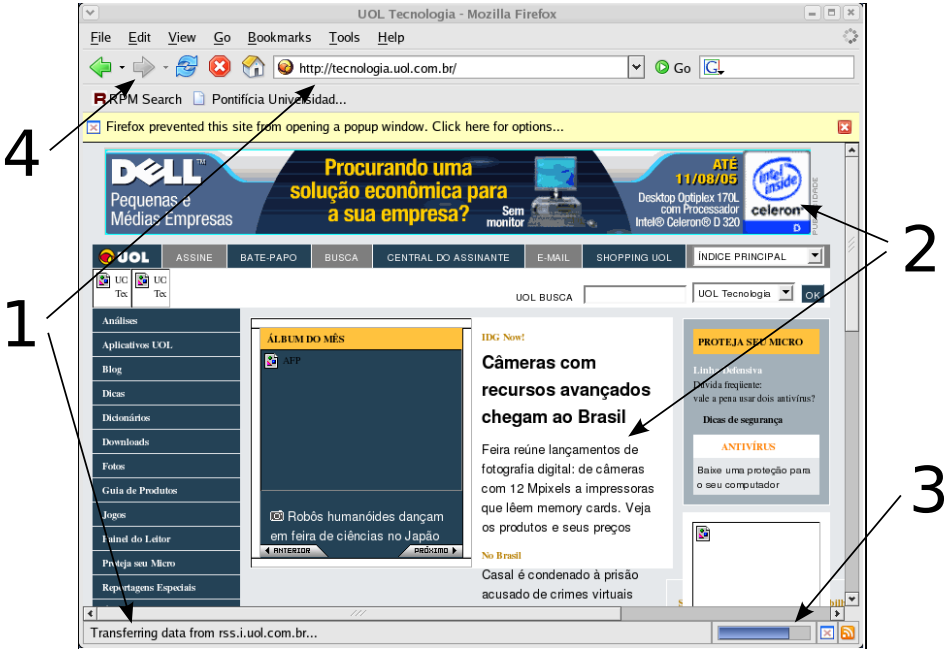


Figura 4.1: Tarefas de um navegador Internet

1. buscar via rede os vários elementos que compõem a página Web;
2. receber, analisar e renderizar o código HTML e os gráficos recebidos;
3. animar os diferentes elementos que compõem a interface do navegador;
4. receber e tratar os eventos do usuário (*clicks*) nos botões do navegador.

Dessa forma, as tarefas definem as atividades a serem realizadas dentro do sistema de computação. Como geralmente há muito mais tarefas a realizar que processadores disponíveis, e as tarefas não têm todas a mesma importância, a gerência de tarefas tem uma grande importância dentro de um sistema operacional.

4.3 A gerência de tarefas

Em um computador, o processador tem de executar todas as tarefas submetidas pelos usuários. Essas tarefas geralmente têm comportamento, duração e importância distintas. Cabe ao sistema operacional organizar as tarefas para executá-las e decidir em que ordem fazê-lo. Nesta seção será estudada a organização básica do sistema de gerência de tarefas e sua evolução histórica.

4.3.1 Sistemas monotarefa

Os primeiros sistemas de computação, nos anos 40, executavam apenas uma tarefa de cada vez. Nestes sistemas, cada programa binário era carregado do disco para a memória e executado até sua conclusão. Os dados de entrada da tarefa eram carregados na memória junto à mesma e os resultados obtidos no processamento eram descarregados de volta no disco após a conclusão da tarefa. Todas as operações de transferência de código e dados entre o disco e a memória eram coordenados por um operador humano. Esses sistemas primitivos eram usados sobretudo para aplicações de cálculo numérico, muitas vezes com fins militares (problemas de trigonometria, balística, mecânica dos fluidos, etc.).

A Figura 4.2 ilustra um sistema desse tipo. Nessa figura, a etapa 1 corresponde à carga do código na memória, a etapa 2 à carga dos dados de entrada na memória, a etapa 3 ao processamento (execução propriamente dita), consumindo dados e produzindo resultados, e a etapa 4 ao término da execução, com a descarga no disco dos resultados de saída produzidos.

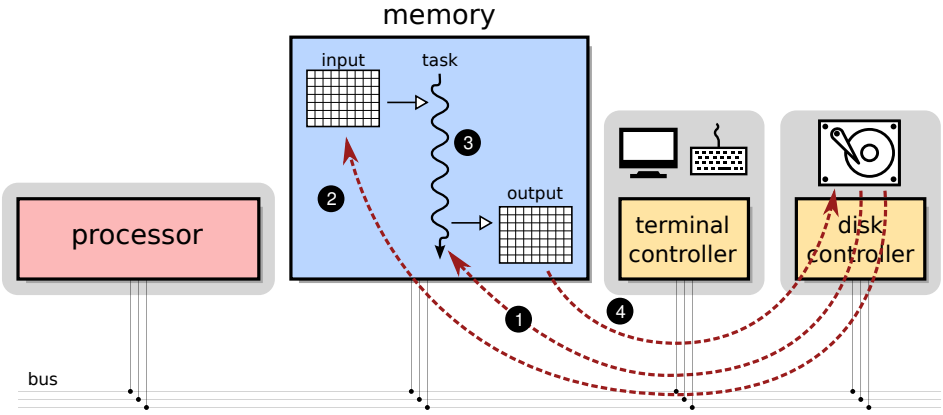


Figura 4.2: Execução de tarefa em um sistema monotarefa.

Nesse método de processamento de tarefas é possível delinear um diagrama de estados para cada tarefa executada pelo sistema, que está representado na Figura 4.3.

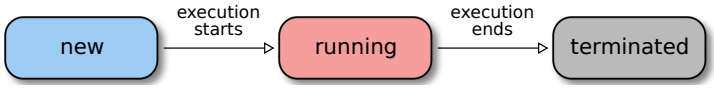


Figura 4.3: Estados de uma tarefa em um sistema monotarefa.

4.3.2 O monitor de sistema

Com a evolução do hardware, as tarefas de carga e descarga de código entre memória e disco, coordenadas por um operador humano, passaram a se tornar críticas: mais tempo era perdido nesses procedimentos manuais que no processamento da tarefa

em si. Para resolver esse problema foi construído um *programa monitor*, que era carregado na memória no início da operação do sistema, com a função de gerenciar a execução dos demais programas. O programa monitor executava continuamente os seguintes passos sobre uma fila de programas a executar, armazenada no disco:

1. carregar um programa do disco para a memória;
2. carregar os dados de entrada do disco para a memória;
3. transferir a execução para o programa recém carregado;
4. aguardar o término da execução do programa;
5. escrever os resultados gerados pelo programa no disco.

Percebe-se claramente que a função do monitor é gerenciar uma fila de programas a executar, mantida no disco. Na medida em que os programas são executados pelo processador, novos programas podem ser inseridos na fila pelo operador do sistema. Além de coordenar a execução dos demais programas, o monitor também colocava à disposição destes uma biblioteca de funções para simplificar o acesso aos dispositivos de hardware (teclado, leitora de cartões, disco, etc.). Assim, o monitor de sistema constitui o precursor dos sistemas operacionais.

4.3.3 Sistemas multitarefas

O uso do monitor de sistema agilizou o uso do processador, mas outros problemas persistiam. Como a velocidade de processamento era muito maior que a velocidade de comunicação com os dispositivos de entrada e saída, o processador ficava ocioso durante os períodos de transferência de informação entre disco e memória[1]. Se a operação de entrada/saída envolvesse fitas magnéticas, o processador podia ficar parado vários minutos, aguardando a tranferência de dados. O custo dos computadores e seu consumo de energia eram elevados demais para deixá-los ociosos por tanto tempo.

A solução encontrada para resolver esse problema foi permitir ao monitor suspender a execução da tarefa que espera dados externos e passar a executar outra tarefa. Mais tarde, quando os dados de que a tarefa suspensa necessita estiverem disponíveis, ela pode ser retomada no ponto onde parou. Para tal, é necessário ter mais memória (para poder carregar mais de um programa ao mesmo tempo) e criar mecanismos no monitor para suspender uma tarefa e retomá-la mais tarde.

Uma forma simples de implementar a suspensão e retomada de tarefas de forma transparente consiste no monitor fornecer um conjunto de rotinas padronizadas de entrada/saída à tarefas; essas rotinas implementadas pelo monitor recebem as solicitações de entrada/saída de dados das tarefas e podem suspender uma execução quando for necessário, devolvendo o controle ao monitor.

A Figura 4.4 ilustra o funcionamento de um sistema multitarefa. Os passos numerados representam a execução (*start*), suspensão (*read*), retomada (*resume*) e conclusão

[1] Essa diferença de velocidades permanece imensa nos sistemas atuais. Por exemplo, em um computador atual a velocidade de acesso à memória é de cerca de 5 nanossegundos ($5 \times 10^{-9} s$), enquanto a velocidade de acesso a dados em um disco rígido SATA é de cerca de 5 milissegundos ($5 \times 10^{-3} s$), ou seja, um milhão de vezes mais lento!

(*exit*) de tarefas pelo monitor. Na figura, a tarefa *A* é suspensa ao solicitar uma leitura de dados, sendo retomada mais tarde, após a execução da tarefa *B*.

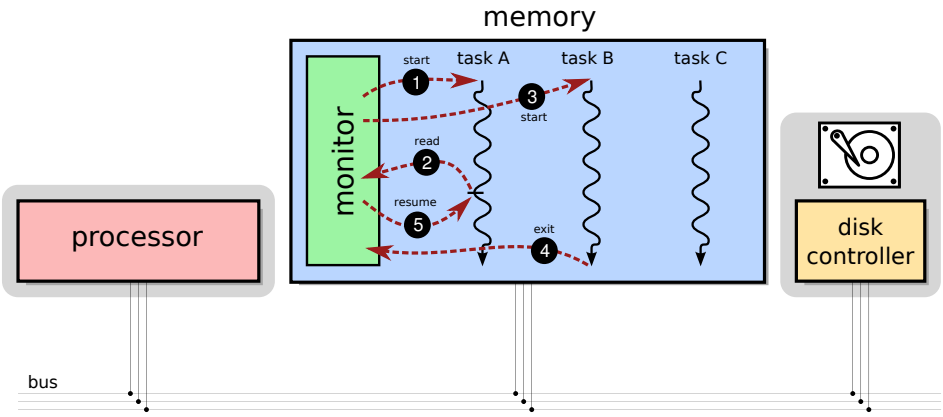


Figura 4.4: Execução de tarefas em um sistema multitarefas.

Essa evolução levou a sistemas mais produtivos (e complexos), nos quais várias tarefas podiam estar em andamento simultaneamente: uma estava ativa (executando) e as demais prontas (esperando pelo processador) ou suspensas (esperando dados ou eventos externos). O diagrama de estados da Figura 4.5 ilustra o comportamento de uma tarefa em um sistema desse tipo:

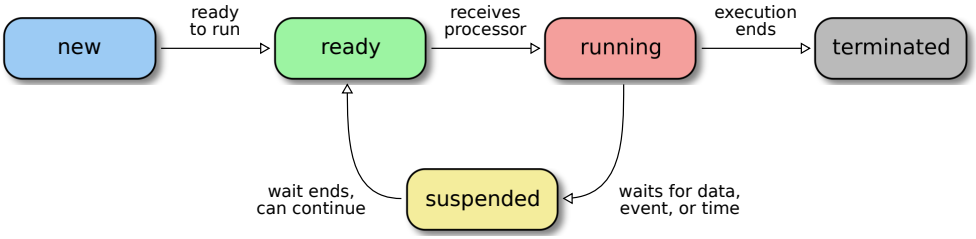


Figura 4.5: Diagrama de estados de uma tarefa em um sistema multitarefas.

4.3.4 Sistemas de tempo compartilhado

Solucionado o problema de evitar a ociosidade do processador, restavam no entanto vários outros problemas a resolver. Por exemplo, o programa a seguir contém um laço infinito; quando uma tarefa executar esse código, ela nunca encerrará nem será suspensa aguardando operações de entrada/saída de dados:

```c
// calcula a soma dos primeiros 1000 inteiros

#include <stdio.h>

int main ()
{
  int i = 0, soma = 0 ;

  while (i <= 1000)
    soma += i ;       // erro: o contador i não foi incrementado

  printf ("A soma vale %d\n", soma);
  exit(0) ;
}
```

Esse tipo de programa podia inviabilizar o funcionamento do sistema, pois a tarefa em execução nunca termina nem solicita operações de entrada/saída, monopolizando o processador e impedindo a execução das demais tarefas (pois o controle nunca volta ao monitor). Além disso, essa solução não era adequada para a criação de aplicações interativas. Por exemplo, a tarefa de um terminal de comandos pode ser suspensa a cada leitura de teclado, perdendo o processador. Se ela tiver de esperar muito para voltar ao processador, a interatividade com o usuário fica prejudicada.

Para resolver essa questão, foi introduzido no início dos anos 60 um novo conceito: o *compartilhamento de tempo*, ou *time-sharing*, através do sistema CTSS – *Compatible Time-Sharing System* [Corbató, 1963]. Nessa solução, para cada tarefa que recebe o processador é definido um prazo de processamento, denominado *fatia de tempo* ou *quantum*[2]. Esgotado seu *quantum*, a tarefa em execução perde o processador e volta para uma fila de tarefas "prontas", que estão na memória aguardando sua oportunidade de executar, e outra tarefa é ativada.

O ato de retirar um recurso "à força" de uma tarefa (neste caso, o processador) é denominado *preempção*. Sistemas que implementam esse conceito são chamados *sistemas preemptivos*. Em um sistema operacional típico, a implementação da preempção por tempo usa as interrupções geradas por um temporizador programável disponível no hardware. Esse temporizador é programado para gerar interrupções em intervalos regulares (a cada milissegundo, por exemplo) que são recebidas por um tratador de interrupção (*interrupt handler*) e encaminhadas ao núcleo; essas ativações periódicas do tratador de interrupção são normalmente chamadas de *ticks*.

Quando uma tarefa recebe o processador, o *núcleo* ajusta um contador de *ticks* que essa tarefa pode usar, ou seja, seu quantum é definido em número de *ticks*. A cada *tick*, esse contador é decrementado; quando ele chegar a zero, a tarefa perde o processador e volta à fila de tarefas prontas. Essa dinâmica de execução está ilustrada na Figura 4.6.

O diagrama de estados das tarefas da Figura 4.5 deve ser portanto reformulado para incluir a preempção por tempo que implementa a estratégia de tempo compartilhado. A Figura 4.7 apresenta esse novo diagrama.

[2]A duração do *quantum* depende muito do tipo de sistema operacional; no Linux ela varia de 10 a 200 milissegundos, dependendo do tipo e prioridade da tarefa [Love, 2010].

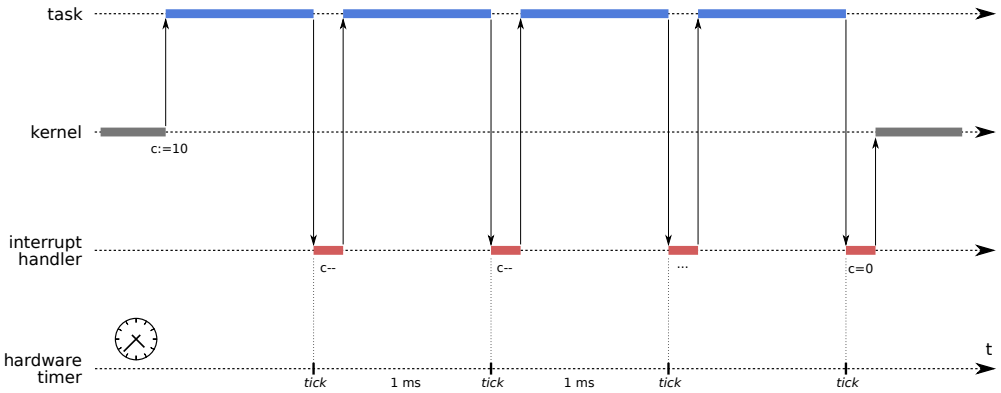


Figura 4.6: Dinâmica da preempção por tempo.

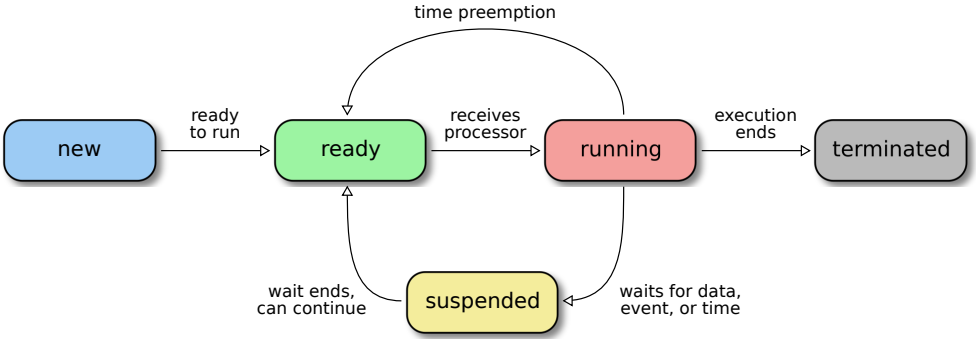


Figura 4.7: Diagrama de estados de uma tarefa em um sistema de tempo compartilhado.

4.4 Ciclo de vida das tarefas

O diagrama apresentado na Figura 4.7 é conhecido na literatura da área como *diagrama de ciclo de vida das tarefas*. Os estados e transições do ciclo de vida têm o seguinte significado:

Nova: A tarefa está sendo criada, i.e. seu código está sendo carregado em memória, junto com as bibliotecas necessárias, e as estruturas de dados do núcleo estão sendo atualizadas para permitir sua execução.

Pronta: A tarefa está em memória, pronta para iniciar ou retomar sua execução, apenas aguardando a disponibilidade do processador. Todas as tarefas prontas são organizadas em uma fila (*fila de prontas*, *ready queue* ou *run queue*), cuja ordem é determinada por algoritmos de escalonamento, que são estudados na Seção 6.

Executando: O processador está dedicado à tarefa, executando suas instruções e fazendo avançar seu estado.

Suspensa: A tarefa não pode executar porque depende de dados externos ainda não disponíveis (do disco ou da rede, por exemplo), aguarda algum tipo de sincronização (o fim de outra tarefa ou a liberação de algum recurso compartilhado) ou simplesmente espera o tempo passar (em uma operação *sleeping*, por exemplo).

Terminada: O processamento da tarefa foi encerrado e ela pode ser removida da memória do sistema.

Tão importantes quanto os estados das tarefas apresentados na Figura 4.7 são as *transições* entre esses estados, que são explicadas a seguir:

$\cdots \rightarrow$ **Nova:** Esta transição ocorre quando uma nova tarefa é admitida no sistema e começa a ser preparada para executar.

Nova → Pronta: ocorre quando a nova tarefa termina de ser carregada em memória, juntamente com suas bibliotecas e dados, estando pronta para executar.

Pronta → Executando: esta transição ocorre quando a tarefa é escolhida pelo escalonador para ser executada (ou para continuar sua execução), dentre as demais tarefas prontas.

Executando → Pronta: esta transição ocorre quando se esgota a fatia de tempo destinada à tarefa (ou seja, o fim do *quantum*); como nesse momento a tarefa não precisa de outros recursos além do processador, ela volta à fila de tarefas prontas até recebê-lo novamente.

Executando → Suspensa: caso a tarefa em execução solicite acesso a um recurso não disponível, como dados externos ou alguma sincronização, ela abandona o processador e fica suspensa até o recurso ficar disponível.

Suspensa → Pronta: quando o recurso solicitado pela tarefa se torna disponível, ela pode voltar a executar, portanto volta ao estado de pronta para aguardar o processador (que pode estar ocupado com outra tarefa).

Executando → Terminada: ocorre quando a tarefa encerra sua execução ou é abortada em consequência de algum erro (acesso inválido à memória, instrução ilegal, divisão por zero, etc.). Na maioria dos sistemas a tarefa que deseja encerrar avisa o sistema operacional através de uma chamada de sistema (no Linux é usada a chamada `exit`).

Terminada → $\cdots$: Uma tarefa terminada é removida da memória e seus registros e estruturas de controle no núcleo são liberados.

A estrutura do diagrama de ciclo de vida das tarefas pode variar de acordo com a interpretação dos autores. Por exemplo, a forma apresentada neste texto condiz com a apresentada em [Silberschatz et al., 2001] e outros autores. Por outro lado, o diagrama apresentado em [Tanenbaum, 2003] divide o estado "suspenso" em dois subestados separados: "bloqueado", quando a tarefa aguarda a ocorrência de algum evento (tempo, entrada/saída, etc.) e "suspenso", para tarefas bloqueadas que foram movidas da memória RAM para a área de troca pelo mecanismo de paginação em disco (vide Capítulo 17).

Todavia, tal distinção de estados não faz mais sentido nos sistemas operacionais atuais baseados em memória paginada, pois neles os processos podem executar mesmo estando somente parcialmente carregados na memória.

Nos sistemas operacionais de mercado é possível consultar o estado das tarefas em execução no sistema. Isso pode ser feito no Windows, por exemplo, através do utilitário "Gerenciador de Tarefas". No Linux, diversos utilitários permitem gerar essa informação, como o comando `top`, cuja saída é mostrada no exemplo a seguir:

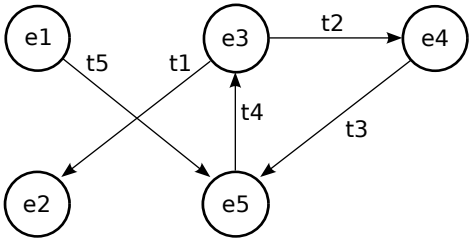
```
top - 16:58:06 up  8:26,  1 user,  load average: 6,04, 2,36, 1,08
Tarefas: 218 total,   7 executando, 211 dormindo,   0 parado,   0 zumbi
%Cpu(s): 49,7 us, 47,0 sy,  0,0 ni,  3,2 id,  0,0 wa,  0,0 hi,  0,1 si,  0,0 st
KiB Mem : 16095364 total,  9856576 free,  3134380 used,  3104408 buff/cache
KiB Swap:        0 total,        0 free,        0 used. 11858380 avail Mem

  PID USUÁRIO   PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
32703 maziero   20   0 2132220 432628 139312 S  44,8  2,7   0:53.64 Web Content
 2192 maziero   20   0 9617080 686444 248996 S  29,8  4,3  20:01.81 firefox
11650 maziero   20   0 2003888 327036 129164 R  24,0  2,0   1:16.70 Web Content
 9844 maziero   20   0 2130164 442520 149508 R  17,9  2,7   1:29.18 Web Content
11884 maziero   20   0   25276   7692   3300 S  15,5  0,0   0:37.18 bash
20425 maziero   20   0   24808   7144   3212 S  14,4  0,0   0:08.39 bash
 1782 maziero   20   0 1788328 235200  77268 S   8,7  1,5  24:12.75 gnome-shell
...
```

Nesse exemplo, existem 218 tarefas no sistema, das quais 7 estão executando (tarefas cuja coluna de Status indica "R" – *running*) e as demais estão suspensas (com Status "S" – *sleeping*). O Linux, como a maioria dos sistemas operacionais, considera que uma tarefa está executando se estiver usando ou esperando por um processador.

Exercícios

1. O que significa *time sharing* e qual a sua importância em um sistema operacional?
2. Como e com base em que critérios é escolhida a duração de um *quantum* de processamento?
3. Considerando o diagrama de estados dos processos apresentado na figura a seguir, complete o diagrama com a transição de estado que está faltando (t_6) e apresente o significado de cada um dos estados e transições.



4. Indique se cada uma das transições de estado de tarefas a seguir definidas é possível ou não. Se a transição for possível, dê um exemplo de situação na qual ela ocorre (*N*: Nova, *P*: pronta, *E*: executando, *S*: suspensa, *T*: terminada).

   - $E \rightarrow P$
   - $E \rightarrow S$
   - $S \rightarrow E$
   - $P \rightarrow N$
   - $S \rightarrow T$
   - $E \rightarrow T$
   - $N \rightarrow S$
   - $P \rightarrow S$

5. Relacione as afirmações abaixo aos respectivos estados no ciclo de vida das tarefas (N: Nova, P: Pronta, E: Executando, S: Suspensa, T: Terminada):

   [ ] O código da tarefa está sendo carregado.
   [ ] A tarefas são ordenadas por prioridades.
   [ ] A tarefa sai deste estado ao solicitar uma operação de entrada/saída.
   [ ] Os recursos usados pela tarefa são devolvidos ao sistema.
   [ ] A tarefa vai a este estado ao terminar seu *quantum*.
   [ ] A tarefa só precisa do processador para poder executar.
   [ ] O acesso a um semáforo em uso pode levar a tarefa a este estado.
   [ ] A tarefa pode criar novas tarefas.
   [ ] Há uma tarefa neste estado para cada processador do sistema.
   [ ] A tarefa aguarda a ocorrência de um evento externo.

Referências

F. Corbató. *The Compatible Time-Sharing System: A Programmer's Guide*. MIT Press, 1963.

R. Love. *Linux Kernel Development, Third Edition*. Addison-Wesley, 2010.

A. Silberschatz, P. Galvin, and G. Gagne. *Sistemas Operacionais – Conceitos e Aplicações*. Campus, 2001.

A. Tanenbaum. *Sistemas Operacionais Modernos, 2ª edição*. Pearson – Prentice-Hall, 2003.

Capítulo 5

Implementação de tarefas

Como visto no capítulo anterior, uma tarefa é a unidade básica de atividade dentro de um sistema operacional. Tarefas podem ser implementadas de várias formas, como processos, *threads*, transações e *jobs*. Neste capítulo são descritos os problemas relacionados à implementação do conceito de tarefa em um sistema operacional típico. São descritas as estruturas de dados necessárias para representar uma tarefa e as operações necessárias para que o processador possa comutar de uma tarefa para outra de forma transparente e eficiente.

5.1 Contextos

Na Seção 4.2 vimos que uma tarefa possui um estado interno bem definido, que representa sua situação atual: a posição de código que ela está executando, os valores de suas variáveis e os recursos que ela utiliza, por exemplo. Esse estado se modifica conforme a execução da tarefa evolui. O estado de uma tarefa em um determinado instante é denominado **contexto**. Uma parte importante do contexto de uma tarefa diz respeito ao estado interno do processador durante sua execução, como o valor do contador de programa (PC - *Program Counter*), do apontador de pilha (SP - *Stack Pointer*) e demais registradores. Além do estado interno do processador, o contexto de uma tarefa também inclui informações sobre os recursos usados por ela, como arquivos abertos, conexões de rede e semáforos.

A cada tarefa presente no sistema é associado um *descritor*, ou seja, uma estrutura de dados no núcleo que representa essa tarefa. Nessa estrutura de dados são armazenadas as informações relativas ao seu contexto e os demais dados necessários à sua gerência, como prioridades, estado, etc. Essa estrutura de dados é geralmente chamada de TCB (do inglês *Task Control Block*) ou PCB (*Process Control Block*). Um TCB tipicamente contém as seguintes informações:

- identificador da tarefa (pode ser um número inteiro, um apontador, uma referência de objeto ou um algum outro identificador);
- estado da tarefa (nova, pronta, executando, suspensa, terminada, etc.);

- informações de contexto do processador (valores contidos nos registradores);
- lista de áreas de memória usadas pela tarefa;
- listas de arquivos abertos, conexões de rede e outros recursos usados pela tarefa (exclusivos ou compartilhados com outras tarefas);
- informações de gerência e contabilização (prioridade, usuário proprietário, data de início, tempo de processamento já decorrido, volume de dados lidos/escritos, etc.).

Dentro do núcleo, os descritores das tarefas são geralmente organizados em listas ou vetores de TCBs. Por exemplo, normalmente há uma lista de tarefas prontas para executar, uma lista de tarefas aguardando acesso ao disco rígido, etc. Para ilustrar concretamente o conceito de TCB, o Apêndice A apresenta o TCB do núcleo Linux em sua versão 1.0.

5.2 Trocas de contexto

Para que o sistema operacional possa suspender e retomar a execução de tarefas de forma transparente (sem que as tarefas o percebam), é necessário definir operações para salvar o contexto atual de uma tarefa em seu TCB e restaurá-lo mais tarde no processador. Por essa razão, o ato de suspender uma tarefa e reativar outra é denominado uma **troca de contexto**.

A implementação da troca de contexto é uma operação delicada, envolvendo a manipulação de registradores e *flags* específicos de cada processador, sendo por essa razão geralmente codificada em linguagem de máquina. No Linux, as operações de troca de contexto para processadores Intel de 64 bits estão definidas nos arquivos `arch/x86/kernel/process_64.c` e `arch/x86/entry/entry_64.S` dos fontes do núcleo (versão 4.*).

Durante uma troca de contexto, existem questões de ordem mecânica e de ordem estratégica a serem resolvidas, o que traz à tona a separação entre mecanismos e políticas já discutida anteriormente (vide Seção 1.2). Por exemplo, o armazenamento e recuperação do contexto e a atualização das informações contidas no TCB de cada tarefa são aspectos mecânicos, providos por um conjunto de rotinas denominado **despachante** ou **executivo** (do inglês *dispatcher*). Por outro lado, a escolha da próxima tarefa a receber o processador a cada troca de contexto é estratégica, podendo sofrer influências de diversos fatores, como as prioridades, os tempos de vida e os tempos de processamento restante de cada tarefa. Essa decisão fica a cargo de um componente de código denominado **escalonador** (*scheduler*, vide Seção 6). Assim, o despachante implementa os mecanismos da gerência de tarefas, enquanto o escalonador implementa suas políticas.

A Figura 5.1 apresenta um diagrama temporal com os principais passos envolvidos em uma troca de contexto:

1. uma tarefa *A* está executando;
2. ocorre uma interrupção do temporizador do hardware e a execução desvia para a rotina de tratamento, no núcleo;

3. a rotina de tratamento ativa o despachante;
4. o despachante salva o estado da tarefa *A* em seu TCB e atualiza suas informações de gerência;
5. opcionalmente, o despachante consulta o escalonador para escolher a próxima tarefa a ativar (*B*);
6. o despachante resgata o estado da tarefa *B* de seu TCB e a reativa.

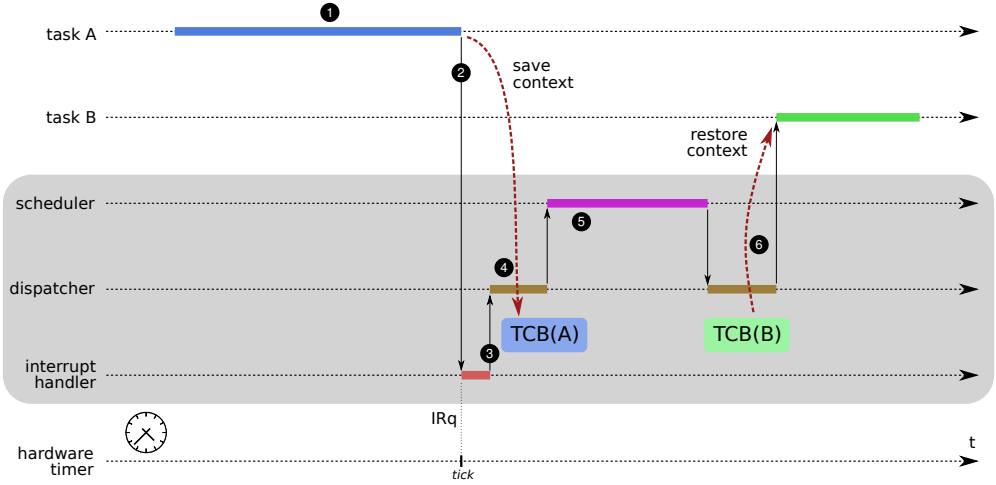


Figura 5.1: Passos de uma troca de contexto.

É importante observar que uma troca de contexto pode ser provocada pelo fim do *quantum* atual (através de uma interrupção de tempo), por um evento ocorrido em um periférico (uma interrupção do respectivo controlador), por uma chamada de sistema emitida pela tarefa corrente (uma interrupção de software) ou até mesmo por algum erro de execução que possa provocar uma exceção no processador.

A frequência de trocas de contexto tem impacto na eficiência do sistema operacional: quanto menor o número de trocas de contexto e menor a duração de cada troca, mais tempo sobrará para a execução das tarefas em si. Assim, é possível definir uma **medida de eficiência** $\mathcal{E}$ do uso do processador, em função das durações médias do *quantum* de tempo t_q e da troca de contexto t_{tc}:

$$\mathcal{E} = \frac{t_q}{t_q + t_{tc}}$$

Por exemplo, um sistema no qual as trocas de contexto duram $100\mu s$ e cujo *quantum* médio é de $10ms$ terá uma eficiência $\mathcal{E} = \frac{10ms}{10ms+100\mu s} = 99\%$. Caso a duração do quantum seja reduzida para $1ms$, a eficiência cairá para $\mathcal{E} = \frac{1ms}{1ms+100\mu s} = 91\%$. A eficiência final da gerência de tarefas é influenciada por vários fatores, como a carga do sistema (mais

tarefas ativas implicam em mais tempo gasto pelo escalonador, aumentando t_{tc}) e o perfil das aplicações (aplicações que fazem muita entrada/saída saem do processador antes do final de seu *quantum*, diminuindo o valor médio de t_q).

Nos sistemas atuais, a realização de uma troca de contexto, envolvendo a interrupção da tarefa atual, o salvamento de seu contexto e a reativação da próxima tarefa, é uma operação relativamente rápida (alguns microssegundos, dependendo do hardware e do sistema operacional). A execução do escalonador, entretanto, pode ser bem mais demorada, sobretudo se houverem muitas tarefas prontas para executar. Por esta razão, muitos sistemas operacionais não executam o escalonador a cada troca de contexto, mas apenas periodicamente, quando há necessidade de reordenar a fila de tarefas prontas. Nesse caso, o despachante sempre ativa a primeira tarefa dessa fila.

5.3 Processos

Há diversas formas de implementar o conceito de tarefa. Uma forma muito empregada pelos sistemas operacionais é o uso de **processos**, apresentado nesta seção.

5.3.1 O conceito de processo

Historicamente, um processo era definido como sendo uma tarefa com seus respectivos recursos, como arquivos abertos e canais de comunicação, em uma área de memória delimitada e isolada das demais. Ou seja, um processo seria uma espécie de "cápsula" isolada de execução, contendo uma tarefa e seus recursos. Essa visão é mantida por muitos autores, como [Silberschatz et al., 2001] e [Tanenbaum, 2003], que apresentam processos como equivalentes a tarefas.

De fato, os sistemas operacionais mais antigos, até meados dos anos 80, suportavam somente um fluxo de execução em cada processo. Assim, as unidades de execução (tarefa) e de recursos (processo) se confundiam. No entanto, quase todos os sistemas operacionais atuais suportam e existência de mais de uma tarefa em cada processo, como é o caso do Linux, Windows, iOS e os sistemas UNIX mais recentes.

Hoje em dia o processo deve ser visto como uma *unidade de contexto*, ou seja, um contêiner de recursos utilizados por uma ou mais tarefas para sua execução: áreas de memória (código, dados, pilha), informações de contexto e descritores de recursos do núcleo (arquivos abertos, conexões de rede, etc). Um processo pode então conter várias tarefas, que compartilham esses recursos. Os processos são isolados entre si pelos mecanismos de proteção providos pelo hardware (isolamento de áreas de memória, níveis de operação e chamadas de sistema), impedindo que uma tarefa do processo p_a acesse um recurso atribuído ao processo p_b. A Figura 5.2 ilustra o conceito de processo, visto como um contêiner de recursos.

Os sistemas operacionais atuais geralmente associam por *default* uma única tarefa a cada processo, o que corresponde à execução de um programa sequencial (iniciado pela função `main()` de um programa em C, por exemplo). Caso se deseje associar mais tarefas ao mesmo processo (para construir o navegador Internet da Figura 4.1, por exemplo), cabe ao desenvolvedor escrever o código necessário para solicitar ao núcleo a criação dessas tarefas adicionais, usualmente sob a forma de *threads* (apresentadas na Seção 5.4).

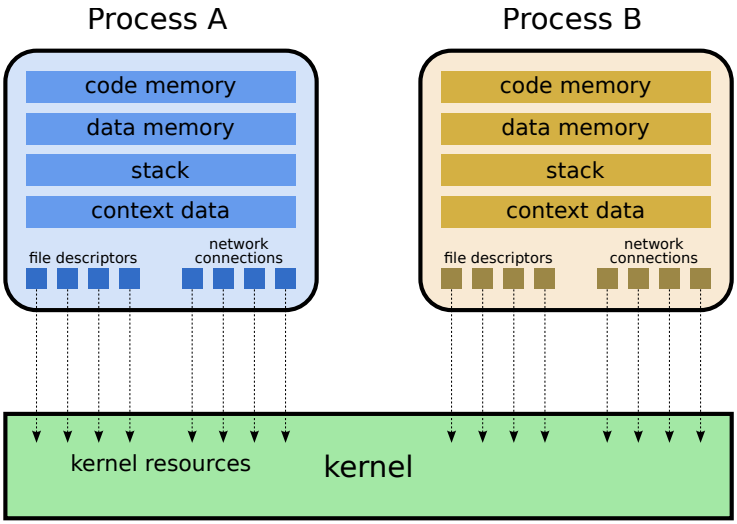


Figura 5.2: O processo visto como um contêiner de recursos.

O núcleo do sistema operacional mantém descritores de processos, denominados PCBs (*Process Control Blocks*), para armazenar as informações referentes aos processos ativos. Um PCB contém informações como o identificador do processo (PID – *Process IDentifier*), seu usuário, prioridade, data de início, caminho do arquivo contendo o código executado pelo processo, áreas de memória em uso, arquivos abertos, etc. A listagem a seguir mostra um conjunto de processos no sistema Linux.

```
top - 16:58:06 up  8:26,  1 user,  load average: 6,04, 2,36, 1,08
Tarefas: 218 total,   7 executando, 211 dormindo,   0 parado,   0 zumbi
%Cpu(s): 49,7 us, 47,0 sy,  0,0 ni,  3,2 id,  0,0 wa,  0,0 hi,  0,1 si,  0,0 st
KiB Mem : 16095364 total,  9856576 free,  3134380 used,  3104408 buff/cache
KiB Swap:        0 total,        0 free,        0 used. 11858380 avail Mem

  PID USUÁRIO   PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
32703 maziero   20   0 2132220 432628 139312 S  44,8  2,7   0:53.64 Web Content
 2192 maziero   20   0 9617080 686444 248996 S  29,8  4,3  20:01.81 firefox
11650 maziero   20   0 2003888 327036 129164 R  24,0  2,0   1:16.70 Web Content
 9844 maziero   20   0 2130164 442520 149508 R  17,9  2,7   1:29.18 Web Content
11884 maziero   20   0   25276   7692   3300 S  15,5  0,0   0:37.18 bash
20425 maziero   20   0   24808   7144   3212 S  14,4  0,0   0:08.39 bash
 1782 maziero   20   0 1788328 235200  77268 S   8,7  1,5  24:12.75 gnome-shell
...
```

Associando-se tarefas a processos, o descritor (TCB) de cada tarefa pode ser bastante simplificado: para cada tarefa, basta armazenar seu identificador, os registradores do processador e uma referência ao processo onde a tarefa executa. Observa-se também que a troca de contexto entre duas tarefas dentro do mesmo processo é muito mais simples e rápida que entre tarefas em processos distintos, pois somente os registradores do

processador precisam ser salvos/restaurados (pois as áreas de memória e demais recursos são comuns a ambas). Essas questões são aprofundadas na Seção 5.4.

5.3.2 Gestão de processos

Durante a vida do sistema, processos são criados e destruídos. Essas operações são disponibilizadas às aplicações através de chamadas de sistema; cada sistema operacional tem suas próprias chamadas para a gestão de processos. O quadro 5.1 traz exemplos de algumas chamadas de sistema usadas na gestão de processos.

Ação	**Windows**	**Linux**
Criar um novo processo	`CreateProcess()`	`fork(), execve()`
Encerrar o processo corrente	`ExitProcess()`	`exit()`
Encerrar outro processo	`TerminateProcess()`	`kill()`
Obter o ID do processo corrente	`GetCurrentProcessId()`	`getpid()`

Tabela 5.1: Chamadas de sistema para a gestão de processos.

No caso dos sistemas UNIX, a criação de novos processos é feita em duas etapas: na primeira etapa, um processo cria uma réplica de si mesmo, usando a chamada de sistema `fork()`. Todo o espaço de memória do processo inicial (pai) é copiado para o novo processo (filho), incluindo o código da(s) tarefa(s) associada(s) e os descritores dos arquivos e demais recursos associados ao mesmo. A Figura 5.3 ilustra o funcionamento dessa chamada.

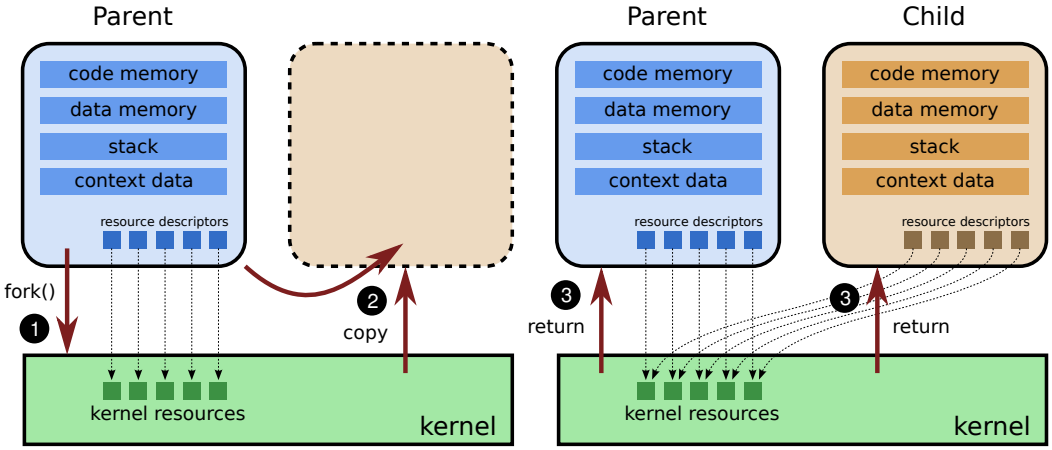


Figura 5.3: Execução da chamada de sistema `fork()`.

A chamada de sistema `fork()` é invocada por um processo, mas dois processos recebem seu retorno: o *processo pai*, que a invocou, e o *processo filho*, que acabou de ser criado e que possui o mesmo estado interno que o pai (ele também está aguardando o retorno da chamada de sistema). Ambos os processos acessam os mesmos recursos do núcleo, embora executem em áreas de memória distintas.

Na segunda etapa, o processo filho usa a chamada de sistema `execve()` para carregar um novo código binário em sua memória. Essa chamada substitui o código do processo que a invoca pelo código executável contido em um arquivo informado como parâmetro. A listagem a seguir apresenta um exemplo de uso conjunto dessas duas chamadas de sistema:

```
1  #include <unistd.h>
2  #include <sys/types.h>
3  #include <sys/wait.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  int main (int argc, char *argv[], char *envp[])
8  {
9     int pid ;                        // identificador de processo
10
11    pid = fork () ;                  // replicação do processo
12
13    if ( pid < 0 )                   // fork funcionou?
14    {
15       perror ("Erro: ") ;           // não, encerra este processo
16       exit (-1) ;
17    }
18    else                             // sim, fork funcionou
19       if ( pid > 0 )                // sou o processo pai?
20          wait (0) ;                 // sim, vou esperar meu filho concluir
21       else                          // não, sou o processo filho
22       {
23          // carrega outro código binário para executar
24          execve ("/bin/date", argv, envp) ;
25          perror ("Erro: ") ;        // execve não funcionou
26       }
27    printf ("Tchau !\n") ;
28    exit(0) ;                        // encerra este processo
29 }
```

A chamada de sistema `exit()` usada no exemplo acima serve para informar ao núcleo do sistema operacional que o processo em questão não é mais necessário e pode ser destruído, liberando todos os recursos a ele associados (arquivos abertos, conexões de rede, áreas de memória, etc.). Processos podem solicitar ao núcleo o encerramento de outros processos, mas essa operação só é aplicável a processos do mesmo usuário, ou se o processo solicitante pertencer ao administrador do sistema.

Outro aspecto importante a ser considerado em relação a processos diz respeito à comunicação. Tarefas associadas ao mesmo processo podem trocar informações facilmente, pois compartilham as mesmas áreas de memória. Todavia, isso não é possível entre tarefas associadas a processos distintos. Para resolver esse problema, o núcleo deve prover às aplicações chamadas de sistema que permitam a comunicação interprocessos (IPC – *Inter-Process Communication*). Esse tema será estudado em profundidade no Capítulo 8.

5.3.3 Hierarquia de processos

Na operação de criação de processos do UNIX aparece de maneira bastante clara a noção de **hierarquia** entre processos: processos são filhos de outros processos. À medida em que processos são criados, forma-se uma *árvore de processos* no sistema, que pode ser usada para gerenciar de forma coletiva os processos ligados à mesma aplicação ou à mesma sessão de trabalho de um usuário, pois irão constituir uma sub-árvore de processos. Por exemplo, quando um processo encerra, seus filhos são informados sobre isso, e podem decidir se também encerram ou se continuam a executar.

Por outro, nos sistemas Windows, todos os processos têm o mesmo nível hierárquico, não havendo distinção entre pais e filhos. O comando `pstree` do sistema Linux permite visualizar a árvore de processos do sistema, como mostra a listagem de exemplo a seguir[1].

```
1  init-+-cron
2       |-dhcpcd
3       |-getty
4       |-getty
5       |-ifplugd
6       |-ifplugd
7       |-lighttpd---php-cgi-+-php-cgi
8       |                    `-php-cgi
9       |-logsave
10      |-logsave
11      |-ntpd
12      |-openvpn
13      |-p910nd
14      |-rsyslogd-+-{rsyslogd}
15      |          `-{rsyslogd}
16      |-sshd---sshd---sshd---pstree
17      |-thd
18      `-udevd-+-udevd
19              `-udevd
```

5.4 Threads

Conforme visto na Seção 5.3, os primeiros sistemas operacionais suportavam uma única tarefa por processo. À medida em que as aplicações se tornavam mais complexas, essa limitação se tornou bem inconveniente. Por exemplo, um editor de textos geralmente executa tarefas simultâneas de edição, formatação, paginação e verificação ortográfica sobre os mesmos dados (o texto em edição). Da mesma forma, processos que implementam servidores de rede (de arquivos, bancos de dados, etc.) devem gerenciar as conexões de vários usuários simultaneamente, que muitas vezes requisitam as mesmas informações. Essas demandas evidenciaram a necessidade de suportar mais de uma tarefa operando sobre os mesmos recursos, ou seja, dentro do mesmo processo.

[1] Listagem obtida de um sistema *Raspberry Pi* com SO Linux *Raspbian* 8.

5.4.1 Definição de *thread*

Uma *thread* é definida como sendo um fluxo de execução independente. Um processo pode conter uma ou mais *threads*, cada uma executando seu próprio código e compartilhando recursos com as demais *threads* localizadas no mesmo processo[2]. Cada *thread* é caracterizada por um código em execução e um pequeno contexto local, o chamado *Thread Local Storage* (TLS), composto pelos registradores do processador e uma área de pilha em memória, para que a *thread* possa armazenar variáveis locais e efetuar chamadas de funções.

Threads são também utilizadas para implementar fluxos de execução dentro do núcleo do SO, neste caso recebendo o nome de *threads de núcleo* (em oposição às *threads* dos processos, denominadas *user threads*). Além de representar as *threads* de usuário dentro do núcleo, as *threads* de núcleo também incluem atividades internas do núcleo, como rotinas de *drivers* de dispositivos ou tarefas de gerência. A Figura 5.4 ilustra o conceito de *threads* de usuário e de núcleo. Na figura, o processo A tem várias *threads*, enquanto o processo B é sequencial (tem uma única *thread*).

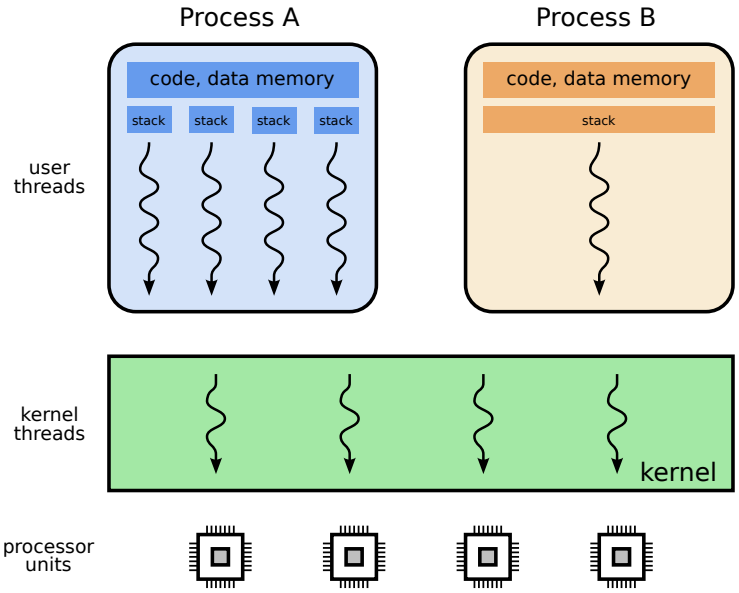


Figura 5.4: *Threads* de usuário e de núcleo.

5.4.2 Modelos de *threads*

As *threads* contidas nos processos, definidas no espaço de usuário, devem ser gerenciadas pelo núcleo do sistema operacional. Essa gerência pode ser feita de diversas formas, conforme os modelos de implementação de *threads* apresentados nesta seção.

[2]Alguns autores usam também o termo *processo leve* (*lightweigth process*) como sinônimo de *thread*.

O modelo N:1

Os sistemas operacionais mais antigos suportavam apenas processos sequenciais, com um único fluxo de execução em cada um. Os desenvolvedores de aplicações contornaram esse problema construindo bibliotecas para salvar, modificar e restaurar os registradores da CPU dentro do processo, permitindo assim criar e gerenciar vários fluxos de execução (*threads*) dentro de cada processo, sem a participação do núcleo.

Com essas bibliotecas, uma aplicação pode lançar várias *threads* conforme sua necessidade, mas o núcleo do sistema irá sempre perceber (e gerenciar) apenas um fluxo de execução dentro de cada processo (ou seja, o núcleo irá manter apenas uma *thread* de núcleo por processo). Esta forma de implementação de *threads* é denominada **Modelo de *Threads* N:1**, pois N *threads* dentro de um processo são mapeadas em uma única *thread* no núcleo. Esse modelo também é denominado *fibers* ou ainda *green threads*. A Figura 5.5 ilustra o modelo N:1.

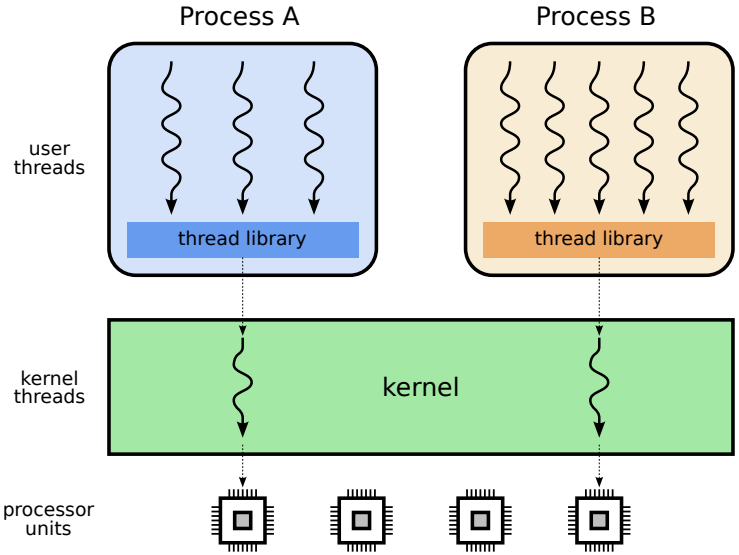


Figura 5.5: O modelo de *threads* N:1.

O modelo de *threads* N:1 é muito utilizado, por ser leve e de fácil implementação. Como o núcleo somente vê uma *thread*, a carga de gerência imposta ao núcleo é pequena e não depende do número de *threads* dentro da aplicação. Essa característica torna este modelo útil na construção de aplicações que exijam muitos *threads*, como jogos ou simulações de grandes sistemas[3]. Exemplos de implementação desse modelo são as bibliotecas *GNU Portable Threads* [Engeschall, 2005], *User-Mode Scheduling* (UMS, Microsoft) e *Green Threads* (Java).

Por outro lado, o modelo de *threads* N:1 apresenta alguns problemas:

[3] A simulação detalhada do tráfego viário de uma cidade grande, por exemplo, pode exigir uma *thread* para cada veículo, podendo portanto chegar a milhões de *threads*.

- as operações de entrada/saída são realizadas pelo núcleo; se uma *thread* de usuário solicitar uma leitura do teclado, por exemplo, a *thread* de núcleo correspondente será suspensa até a conclusão da operação, bloqueando todas as *threads* daquele processo;
- o núcleo do sistema divide o tempo de processamento entre as *threads* de núcleo. Assim, um processo com 100 *threads* irá receber o mesmo tempo de processador que outro com apenas uma *thread*, ou seja, cada *thread* do primeiro processo irá receber 1/100 do tempo que recebe a *thread* única do outro processo;
- *threads* do mesmo processo não podem executar em paralelo, mesmo se o computador dispuser de vários processadores ou *cores*, porque o núcleo somente escalona as *threads* de núcleo nos processadores.

O modelo 1:1

A necessidade de suportar aplicações *multithread* levou os desenvolvedores de sistemas operacionais a incorporar a gerência dos *threads* dos processo no núcleo do sistema. Para cada *thread* de usuário foi então associado um *thread* correspondente dentro do núcleo, suprimindo com isso a necessidade de bibliotecas de *threads*. Essa forma de implementação é denominada **Modelo de *Threads* 1:1**, sendo apresentada na Figura 5.6. Este é o modelo mais frequente nos sistemas operacionais atuais, como o Windows NT e seus descendentes, além da maioria dos UNIXes.

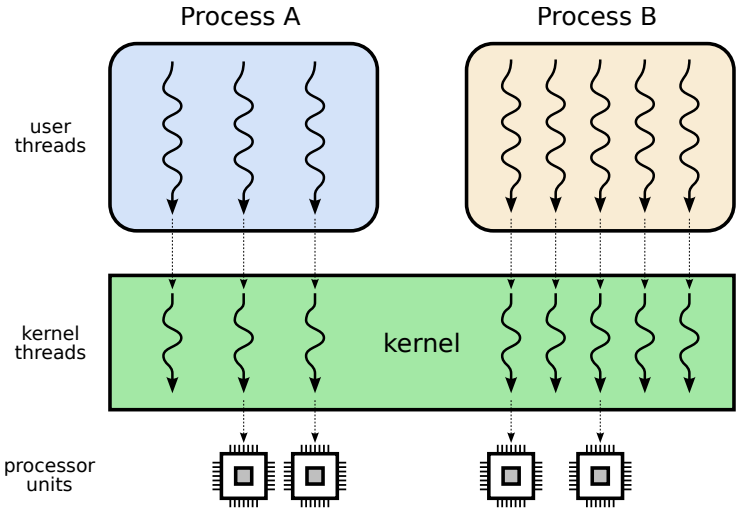


Figura 5.6: O modelo de *threads* 1:1.

O modelo 1:1 resolve vários problemas: caso uma *thread* de usuário solicite uma operação bloqueante, somente sua *thread* de núcleo correspondente será suspensa, sem afetar as demais *threads* do processo. Além disso, a distribuição de processamento entre as *threads* é mais justa e, caso o hardware tenha mais de um processador, mais *threads* do mesmo processo podem executar ao mesmo tempo, o que não era possível no modelo N:1.

O modelo de *threads* 1:1 é adequado para a maioria das situações usuais e atende bem às necessidades das aplicações interativas e servidores de rede. No entanto, é pouco escalável: a criação de um número muito grande de *threads* impõe uma carga elevada ao núcleo do sistema, inviabilizando aplicações com muitas tarefas (como grandes servidores Web e simulações de grande porte).

O modelo N:M

Para resolver o problema de escalabilidade da abordagem 1:1, alguns sistemas operacionais implementam um modelo híbrido, que agrega características dos modelos anteriores. Nesse novo modelo, uma biblioteca gerencia um conjunto de N *threads* de usuário (dentro do processo), que é mapeado em $M < N$ *threads* no núcleo. Essa abordagem, denominada **Modelo de *Threads* N:M**, é apresentada na Figura 5.7.

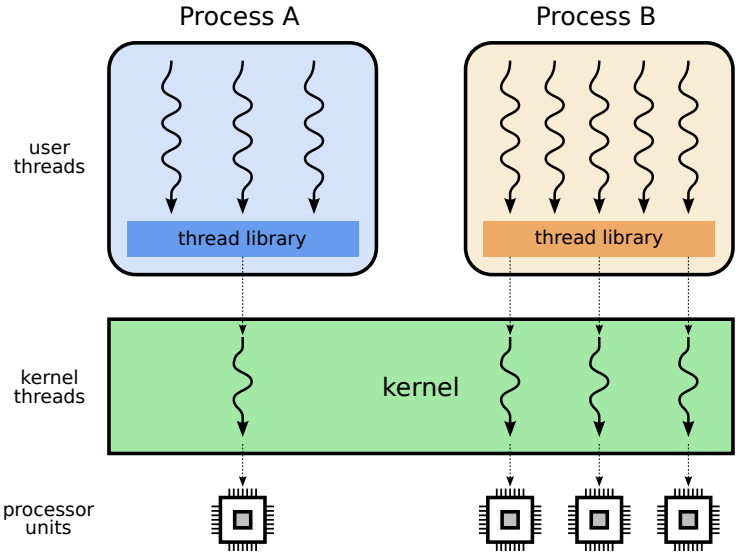


Figura 5.7: O modelo de *threads* N:M.

O conjunto de *threads* de núcleo associadas a um processo, denominado *thread pool*, geralmente contém uma *thread* para cada *thread* de usuário bloqueada, mais uma *thread* para cada processador disponível; esse conjunto pode ser ajustado dinamicamente, conforme a necessidade da aplicação.

O modelo N:M é implementado pelo Solaris e também pelo projeto KSE (*Kernel-Scheduled Entities*) do FreeBSD [Evans and Elischer, 2003] baseado nas ideias apresentadas em [Anderson et al., 1992]. Ele alia as vantagens de maior interatividade do modelo 1:1 à maior escalabilidade do modelo N:1. Como desvantagens desta abordagem podem ser citadas sua complexidade de implementação e maior custo de gerência dos *threads* de núcleo, quando comparados ao modelo 1:1.

Comparação entre os modelos

A Tabela 5.2 resume os principais aspectos dos três modelos de implementação de *threads* e faz um comparativo entre eles.

Modelo	**N:1**	**1:1**	**N:M**
Resumo	N *threads* do processo mapeados em uma *thread* de núcleo	Cada *thread* do processo mapeado em uma *thread* de núcleo	N *threads* do processo mapeados em M< N *threads* de núcleo
Implementação	no processo (biblioteca)	no núcleo	em ambos
Complexidade	baixa	média	alta
Custo de gerência	baixo	médio	alto
Escalabilidade	alta	baixa	alta
Paralelismo entre *threads* do mesmo processo	não	sim	sim
Troca de contexto entre *threads* do mesmo processo	rápida	lenta	rápida
Divisão de recursos entre tarefas	injusta	justa	variável, pois o mapeamento *thread*→ processador é dinâmico
Exemplos	GNU Portable Threads, Microsoft UMS	Windows, Linux	Solaris, FreeBSD KSE

Tabela 5.2: Comparativo dos modelos de *threads*.

5.4.3 Programando com *threads*

No passado, cada sistema operacional definia sua própria interface para a criação de *threads*, o que levou a problemas de portabilidade das aplicações. Em 1995 foi definido o padrão *IEEE POSIX 1003.1c*, mais conhecido como *POSIX Threads* ou simplesmente *PThreads* [Nichols et al., 1996], que define uma interface padronizada para o uso de *threads* na linguagem C. Esse padrão é amplamente difundido e suportado hoje em dia. A listagem a seguir, extraída de [Barney, 2005], exemplifica o uso do padrão *PThreads*:

```
1  // Exemplo de uso de threads Posix em C no Linux
2  // Compilar com gcc exemplo.c -o exemplo -lpthread
3
4  #include <pthread.h>
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <unistd.h>
8
9  #define NUM_THREADS 5
10
11 // cada thread vai executar esta função
12 void *print_hello (void *threadid)
13 {
14    printf ("%ld: Hello World!\n", (long) threadid);
15    sleep (5) ;
16    printf ("%ld: Bye bye World!\n", (long) threadid);
17    pthread_exit (NULL);     // encerra esta thread
18 }
```

```
19 // thread "main" (vai criar as demais threads)
20 int main (int argc, char *argv[])
21 {
22    pthread_t thread[NUM_THREADS];
23    long status, i;
24
25    // cria as demais threads
26    for(i = 0; i < NUM_THREADS; i++)
27    {
28       printf ("Creating thread %ld\n", i);
29       status = pthread_create (&thread[i], NULL, print_hello, (void *) i);
30
31       if (status) // ocorreu um erro
32       {
33          perror ("pthread_create");
34          exit (-1);
35       }
36    }
37
38    // encerra a thread "main"
39    pthread_exit (NULL);
40 }
```

Nessa listagem pode-se perceber que o programa inicia sua execução com apenas uma *thread* (representada pela função `main()`, linha 20); esta, por sua vez, cria as demais *threads* (linha 29) e indica o código a ser executado pelas mesmas (no exemplo, todas executam a mesma função `print_hello()`, linha 12).

Threads podem ser utilizadas em diversas outras linguagens de programação, como Java, Python, Perl, etc. O código a seguir traz um exemplo simples de criação de *threads* em Java (extraído da documentação oficial da linguagem):

```
1  // save as MyThread.java; javac MyThread.java; java MyThread
2
3  public class MyThread extends Thread {
4     int threadID;
5
6     private static final int NUM_THREADS = 5 ;
7
8     MyThread (int ID) {
9        threadID = ID;
10    }
11
12    // corpo de cada thread
13    public void run () {
14       System.out.println (threadID + ": Hello World!") ;
15       try {
16         Thread.sleep(5000);
17       }
18       catch (InterruptedException e) {
19         e.printStackTrace();
20       }
21       System.out.println (threadID + ": Bye bye World!") ;
22    }
```

```
23    public static void main (String args[]) {
24
25       MyThread[] t = new MyThread[NUM_THREADS] ;
26
27       // cria as threads
28       for (int i = 0; i < NUM_THREADS; i++) {
29          t[i] = new MyThread (i);
30       }
31
32       // inicia a execução das threads
33       for (int i = 0; i < NUM_THREADS; i++) {
34          t[i].start () ;
35       }
36    }
37 }
```

5.5 Uso de processos *versus threads*

Neste capítulo foram apresentadas duas formas de implementar tarefas em um sistema operacional: os processos e as *threads*. No caso do desenvolvimento de sistemas complexos, compostos por várias tarefas interdependentes executando em paralelo, qual dessas formas de implementação deve ser escolhida? Um navegador de Internet (*browser*), um servidor Web ou um servidor de bancos de dados atendendo vários usuários simultâneos são exemplos desse tipo de sistema.

A implementação de sistemas usando **um processo para cada tarefa** é uma possibilidade. Essa abordagem tem como grande vantagem a robustez, pois um erro ocorrido em um processo ficará restrito ao seu espaço de memória pelos mecanismos de

isolamento do hardware. Além disso, os processos podem ser configurados para executar com usuários (e permissões) distintas, aumentando a segurança do sistema. Por outro lado, nessa abordagem o compartilhamento de dados entre os diversos processos que compõem a aplicação irá necessitar de mecanismos especiais, como áreas de memória compartilhada, pois um processo não pode acessar as áreas de memória dos demais processos. Como exemplos de sistemas que usam a abordagem baseada em processos, podem ser citados o servidor web Apache (nas versões 1.*) e o servidor de bancos de dados PostgreSQL.

Também é possível implementar todas as tarefas em um único processo, usando **uma *thread* para cada tarefa**. Neste caso, todas as *threads* compartilham o mesmo espaço de endereçamento e os mesmos recursos (arquivos abertos, conexões de rede, etc), tornando mais simples implementar a interação entre as tarefas. Além disso, a execução é mais ágil, pois é mais rápido criar uma *thread* que um processo. Todavia, um erro em uma *thread* pode se alastrar às demais, pondo em risco a robustez da aplicação inteira. Muitos sistemas usam uma abordagem multi-*thread*, como o servidor Web IIS (Microsoft *Internet Information Server*) e o editor de textos LibreOffice.

Sistemas mais recentes e sofisticados usam uma abordagem híbrida, com o **uso conjunto de processos e *threads***. A aplicação é composta por vários processos, cada um contendo diversas *threads*. Busca-se com isso aliar a robustez proporcionada pelo isolamento entre processos e o desempenho, menor consumo de recursos e facilidade de programação proporcionados pelas *threads*. Esse modelo híbrido é usado, por exemplo, nos navegadores Chrome e Firefox: cada aba de navegação é atribuída a um novo processo, que cria as *threads* necessárias para buscar e renderizar aquele conteúdo e interagir com o usuário. O servidor Web Apache 2.* e o servidor de bases de dados Oracle também usam essa abordagem.

A Figura 5.8 ilustra as três abordagens apresentadas nesta seção; a Tabela 5.3 sintetiza a comparação entre elas.

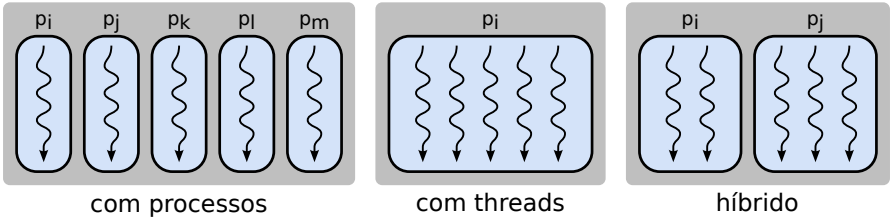


Figura 5.8: Implementação de sistemas usando processos, *threads* ou híbrido.

Característica	Com processos	Com *threads* (1:1)	Híbrido
Custo de criação de tarefas	alto	baixo	médio
Troca de contexto	lenta	rápida	variável
Uso de memória	alto	baixo	médio
Compartilhamento de dados entre tarefas	canais de comunicação e áreas de memoria compartilhada.	variáveis globais e dinâmicas.	ambos.
Robustez	alta, um erro fica contido no processo.	baixa, um erro pode afetar todas as *threads*.	média, um erro pode afetar as *threads* no mesmo processo.
Segurança	alta, cada processo pode executar com usuários e permissões distintas.	baixa, todas as *threads* herdam as permissões do processo onde executam.	alta, *threads* que necessitam as mesmas permissões podem ser agrupadas em um mesmo processo.
Exemplos	Servidor Apache (versões 1.*), SGBD PostGres	Servidor Apache (versões 2.*), SGBD MySQL	Navegadores Chrome e Firefox, SGBD Oracle

Tabela 5.3: Comparativo entre uso de processos e de *threads*.

Exercícios

1. Explique o que é, para que serve e o que contém um TCB - *Task Control Block*.

2. Desenhe o diagrama de tempo da execução do código a seguir, informe qual a saída do programa na tela (com os valores de *x*) e calcule a duração aproximada de sua execução.

```
1  int main()
2  {
3     int x = 0 ;
4
5     fork () ;
6     x++ ;
7     sleep (5) ;
8     wait (0) ;
9     fork () ;
10    wait (0) ;
11    sleep (5) ;
12    x++ ;
13    printf ("Valor de x: %d\n", x) ;
14 }
```

3. Indique quantas letras "X" serão impressas na tela pelo programa abaixo quando for executado com a seguinte linha de comando:

```
a.out 4 3 2 1
```

O comando `a.out` resulta da compilação do programa a seguir:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
   pid_t pid[10];
   int i;

   int N = atoi(argv[argc-2]);

   for (i=0; i<N; i++)
      pid[i] = fork();
   if (pid[0] != 0 && pid[N-1] != 0)
      pid[N] = fork();
   printf("X");
   return 0;
}
```

4. O que são *threads* e para que servem?

5. Quais as principais vantagens e desvantagens de *threads* em relação a processos?

6. Forneça dois exemplos de problemas cuja implementação *multi-thread* não tem desempenho melhor que a respectiva implementação sequencial.

7. Associe as afirmações a seguir aos seguintes modelos de *threads*: a) *many-to-one* (N:1); b) *one-to-one* (1:1); c) *many-to-many* (N:M):

   (a) Tem a implementação mais simples, leve e eficiente.
   (b) Multiplexa os *threads* de usuário em um pool de *threads* de núcleo.
   (c) Pode impor uma carga muito pesada ao núcleo.
   (d) Não permite explorar a presença de várias CPUs pelo mesmo processo.
   (e) Permite uma maior concorrência sem impor muita carga ao núcleo.
   (f) Geralmente implementado por bibliotecas.
   (g) É o modelo implementado no Windows NT e seus sucessores.
   (h) Se um *thread* bloquear, todos os demais têm de esperar por ele.
   (i) Cada *thread* no nível do usuário tem sua correspondente dentro do núcleo.
   (j) É o modelo com implementação mais complexa.

8. Considerando as implementações de *threads* N:1 e 1:1 para o trecho de código a seguir, a) desenhe os diagramas de execução, b) informe as durações aproximadas de execução e c) indique a saída do programa na tela. Considere a operação `sleep()` como uma chamada de sistema (*syscall*).

A chamada `thread_create` cria uma nova *thread*, `thread_exit` encerra a *thread* corrente e `thread_join` espera o encerramento da *thread* informada como parâmetro.

```
1  int y = 0 ;
2
3  void threadBody
4  {
5     int x = 0 ;
6     sleep (10) ;
7     printf ("x: %d, y:%d\n", ++x, ++y) ;
8     thread_exit();
9  }
10
11 main ()
12 {
13    thread_create (&tA, threadBody, ...) ;
14    thread_create (&tB, threadBody, ...) ;
15    sleep (1) ;
16    thread_join (&tA) ;
17    thread_join (&tB) ;
18    sleep (1) ;
19    thread_create (&tC, threadBody, ...) ;
20    thread_join (&tC) ;
21 }
```

Referências

T. Anderson, B. Bershad, E. Lazowska, and H. Levy. Scheduler activations: effective kernel support for the user-level management of parallelism. *ACM Transactions on Computer Systems*, 10(1):53–79, February 1992.

B. Barney. POSIX threads programming. http://www.llnl.gov/computing/tutorials/pthreads, 2005.

R. Engeschall. The GNU Portable Threads. http://www.gnu.org/software/pth, 2005.

J. Evans and J. Elischer. Kernel-scheduled entities for FreeBSD. http://www.aims.net.au/chris/kse, 2003.

B. Nichols, D. Buttlar, and J. Farrell. *PThreads Programming*. O'Reilly Media, Inc, 1996.

A. Silberschatz, P. Galvin, and G. Gagne. *Sistemas Operacionais – Conceitos e Aplicações*. Campus, 2001.

A. Tanenbaum. *Sistemas Operacionais Modernos, 2ª edição*. Pearson – Prentice-Hall, 2003.

Capítulo 6

Escalonamento de tarefas

Um dos componentes mais importantes da gerência de tarefas é o **escalonador de tarefas** (*task scheduler*), que decide a ordem de execução das tarefas prontas. O algoritmo utilizado no escalonador define o comportamento do sistema operacional, permitindo obter sistemas que tratem de forma mais eficiente e rápida as tarefas a executar, que podem ter características diversas: aplicações interativas, processamento de grandes volumes de dados, programas de cálculo numérico, etc.

6.1 Tipos de tarefas

Antes de se definir o algoritmo usado por um escalonador de tarefas, é necessário ter em mente a natureza das tarefas que o sistema irá executar. Existem vários critérios que definem o comportamento de uma tarefa; uma primeira classificação possível diz respeito ao seu comportamento temporal:

Tarefas de tempo real: exigem previsibilidade em seus tempos de resposta aos eventos externos, pois geralmente estão associadas ao controle de sistemas críticos, como processos industriais, tratamento de fluxos multimídia, etc. O escalonamento de tarefas de tempo real é um problema complexo, fora do escopo deste livro e discutido mais profundamente em [Burns and Wellings, 1997; Farines et al., 2000].

Tarefas interativas: são tarefas que recebem eventos externos (do usuário ou através da rede) e devem respondê-los rapidamente, embora sem os requisitos de previsibilidade das tarefas de tempo real. Esta classe de tarefas inclui a maior parte das aplicações dos sistemas *desktop* (editores de texto, navegadores Internet, jogos) e dos servidores de rede (e-mail, web, bancos de dados).

Tarefas em lote (*batch*): são tarefas sem requisitos temporais explícitos, que normalmente executam sem intervenção do usuário, como procedimentos de *backup*, varreduras de antivírus, cálculos numéricos longos, tratamentos de grandes massas de dados em lote, renderização de animações, etc.

Além dessa classificação, as tarefas também podem ser classificadas de acordo com seu comportamento no uso do processador:

Tarefas orientadas a processamento (*CPU-bound tasks*): são tarefas que usam intensivamente o processador na maior parte de sua existência. Essas tarefas passam a maior parte do tempo nos estados *pronta* ou *executando*. A conversão de arquivos de vídeo e outros processamentos longos (como os feitos pelo projeto *SETI@home* [Anderson et al., 2002]) são bons exemplos desta classe de tarefas.

Tarefas orientadas a entrada/saída (*IO-bound tasks*): são tarefas que dependem muito mais dos dispositivos de entrada/saída que do processador. Essas tarefas ficam boa parte de suas existências no estado *suspenso*, aguardando respostas às suas solicitações de leitura e/ou escrita de dados nos dispositivos de entrada/saída. Exemplos desta classe de tarefas incluem editores, compiladores e servidores de rede.

É importante observar que uma tarefa pode mudar de comportamento ao longo de sua execução. Por exemplo, um conversor de arquivos de áudio WAV→MP3 alterna constantemente entre fases de processamento e de entrada/saída, até concluir a conversão dos arquivos desejados.

6.2 Objetivos e métricas

Ao se definir um algoritmo de escalonamento, deve-se ter em mente seu objetivo. Todavia, os objetivos do escalonador são muitas vezes contraditórios; o desenvolvedor do sistema tem de escolher o que priorizar, em função do perfil das aplicações a suportar. Por exemplo, um sistema interativo voltado à execução de jogos exige valores de quantum baixos, para que cada tarefa pronta receba rapidamente o processador (provendo maior interatividade). Todavia, valores pequenos de quantum implicam em uma menor eficiência $\mathcal{E}$ no uso do processador, conforme visto na Seção 5.2. Vários critérios podem ser definidos para a avaliação de escalonadores; os mais frequentemente utilizados são:

Tempo de execução (ou de vida) (*turnaround time*, t_t): diz respeito ao tempo total da execução de uma tarefa, ou seja, o tempo decorrido entre a criação da tarefa e seu encerramento, computando todos os tempos de processamento e de espera. É uma medida típica de sistemas em lote, nos quais não há interação direta com os usuários do sistema. Não deve ser confundido com o **tempo de processamento** (t_p), que é o tempo total de uso de processador demandado pela tarefa.

Tempo de espera (*waiting time*, t_w): é o tempo total perdido pela tarefa na fila de tarefas prontas, aguardando o processador. Deve-se observar que esse tempo não inclui os tempos de espera em operações de entrada/saída (que são inerentes à aplicação e aos dispositivos).

Tempo de resposta (*response time*, t_r): é o tempo decorrido entre a chegada de um evento ao sistema e o resultado imediato de seu processamento. Por exemplo, em um editor de textos seria o tempo decorrido entre apertar uma tecla e o caractere

correspondente aparecer na tela. Essa medida de desempenho é típica de sistemas interativos, como sistemas *desktop* e de tempo real; ela depende sobretudo da rapidez no tratamento das interrupções de hardware pelo núcleo e do valor do *quantum* de tempo, para permitir que as tarefas interativas cheguem mais rápido ao processador quando saem do estado suspenso.

Justiça: este critério diz respeito à distribuição do processador entre as tarefas prontas: duas tarefas de comportamento e prioridade similares devem ter durações de execução similares.

Eficiência: a eficiência $\mathcal{E}$, conforme definido na Seção 5.2, indica o grau de utilização do processador na execução das tarefas do usuário. Ela depende sobretudo da rapidez da troca de contexto e da quantidade de tarefas orientadas a entrada/saída no sistema (tarefas desse tipo geralmente abandonam o processador antes do fim do *quantum*, gerando assim mais trocas de contexto que as tarefas orientadas a processamento).

6.3 Escalonamento preemptivo e cooperativo

O escalonador de um sistema operacional pode ser preemptivo ou cooperativo (não-preemptivo):

Sistemas preemptivos: nestes sistemas uma tarefa pode perder o processador caso termine seu *quantum* de tempo, caso execute uma chamada de sistema ou caso ocorra uma interrupção que acorde uma tarefa mais prioritária (que estava suspensa aguardando um evento). A cada interrupção, exceção ou chamada de sistema, o escalonador reavalia todas as tarefas da fila de prontas e decide se mantém ou substitui a tarefa atualmente em execução.

Sistemas cooperativos: a tarefa em execução permanece no processador tanto quanto possível, só liberando o mesmo caso termine de executar, solicite uma operação de entrada/saída ou libere explicitamente o processador[1], voltando à fila de tarefas prontas. Esses sistemas são chamados de *cooperativos* por exigir a cooperação das tarefas entre si na gestão do processador, para que todas possam executar.

Atualmente a maioria dos sistemas operacionais de uso geral é preemptiva. Sistemas mais antigos, como o Windows 3.*, PalmOS 3 e MacOS 8 e 9 operavam de forma cooperativa.

Em um sistema preemptivo simples, normalmente as tarefas só são interrompidas quando o processador está no modo usuário; a *thread* de núcleo correspondente a cada tarefa não sofre interrupções. Entretanto, os sistemas mais sofisticados implementam a preempção de tarefas também no modo núcleo. Essa funcionalidade é importante para sistemas de tempo real, pois permite que uma tarefa de alta prioridade chegue mais rapidamente ao processador quando for reativada. Núcleos de sistema que oferecem essa possibilidade são denominados **núcleos preemptivos**; Solaris, Linux 2.6 e Windows NT são exemplos de núcleos preemptivos.

[1] Uma tarefa pode liberar explicitamente o processador e voltar à fila de prontas através de uma chamada de sistema específica, como `sched_yield()` em Linux ou `SwitchToThread()` em Windows.

6.4 Algoritmos de escalonamento de tarefas

Esta seção descreve os algoritmos mais simples de escalonamento de tarefas encontrados na literatura. Esses algoritmos raramente são usados *ipsis litteris*, mas servem de base conceitual para a construção dos escalonadores mais complexos que são usados em sistemas operacionais reais. Para a descrição do funcionamento de cada algoritmo será considerado um sistema monoprocessado e um conjunto hipotético de 4 tarefas ($t_1 \cdots t_5$) na fila de prontas do sistema operacional, descritas na Tabela 6.1 a seguir.

Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

Tabela 6.1: Tarefas na fila de prontas.

Para simplificar a análise dos algoritmos, as tarefas $t_1 \cdots t_5$ são orientadas a processamento, ou seja, não param para realizar operações de entrada/saída. Cada tarefa tem uma data de ingresso (instante em que entrou no sistema), uma duração (tempo de processamento que necessita para realizar sua execução) e uma prioridade (usada nos algoritmos PRIOc e PRIOp, seção 6.4.5).

6.4.1 First-Come, First Served (FCFS)

A forma de escalonamento mais elementar consiste em simplesmente atender as tarefas em sequência, à medida em que elas se tornam prontas (ou seja, conforme sua ordem de ingresso na fila de tarefas prontas). Esse algoritmo é conhecido como FCFS – *First Come - First Served* – e tem como principal vantagem sua simplicidade.

O diagrama da Figura 6.1 mostra o escalonamento das tarefas de Tabela 6.1 usando o algoritmo FCFS cooperativo (ou seja, sem *quantum* ou outras interrupções). Os quadros sombreados representam o uso do processador (observe que em cada instante apenas uma tarefa ocupa o processador). Os quadros brancos representam as tarefas que já ingressaram no sistema e estão aguardando o processador (tarefas prontas).

Calculando o tempo médio de execução (T_t, a média de $t_t(t_i)$) e o tempo médio de espera (T_w, a média de $t_w(t_i)$) para a execução da Figura 6.1, temos:

$$\begin{aligned} T_t &= \frac{t_t(t_1) + \cdots + t_t(t_5)}{5} = \frac{(5-0) + (7-0) + (11-1) + (12-3) + (14-5)}{5} \\ &= \frac{5+7+10+9+9}{5} = \frac{40}{5} = 8,0s \end{aligned}$$

$$\begin{aligned} T_w &= \frac{t_w(t_1) + \cdots + t_w(t_5)}{5} = \frac{(0-0) + (5-0) + (7-1) + (11-3) + (12-5)}{5} \\ &= \frac{0+5+6+8+7}{5} = \frac{26}{5} = 5,2s \end{aligned}$$

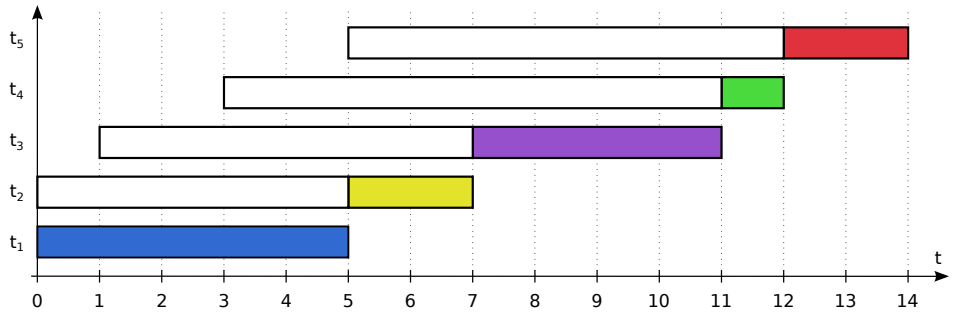


Figura 6.1: Escalonamento FCFS.

6.4.2 Round-Robin (RR)

A adição da preempção por tempo ao escalonamento FCFS dá origem a outro algoritmo de escalonamento bastante popular, conhecido como **escalonamento por revezamento**, ou *Round-Robin*. Considerando as tarefas definidas na tabela anterior e um quantum $t_q = 2s$, seria obtida a sequência de escalonamento apresentada na Figura 6.2.

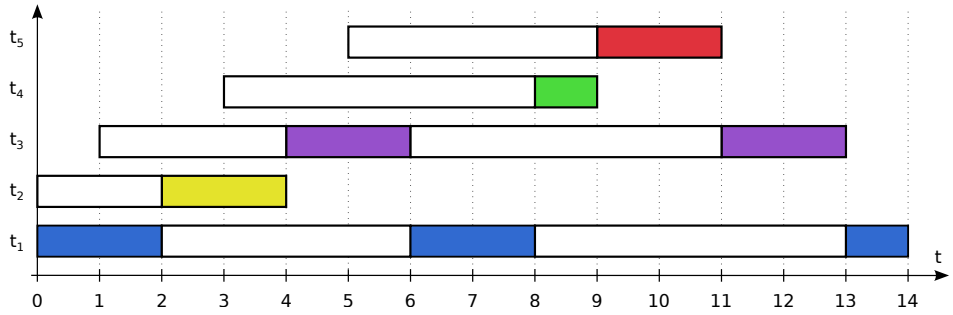


Figura 6.2: Escalonamento *Round-Robin*.

Na Figura 6.2, é importante observar que a execução das tarefas não obedece uma sequência óbvia como $t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_4 \rightarrow t_5 \rightarrow t_1 \rightarrow t_2 \rightarrow \ldots$, mas uma sequência bem mais complexa: $t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_1 \rightarrow t_4 \rightarrow t_5 \rightarrow \ldots$. Isso ocorre por causa da ordem das tarefas na fila de tarefas prontas. Por exemplo, a tarefa t_1 para de executar e volta à fila de tarefas prontas no instante $t = 2$, antes de t_4 ter entrado no sistema (em $t = 3$). Por isso, t_1 retorna ao processador antes de t_4 (em $t = 6$). A Figura 6.3 detalha a evolução da fila de tarefas prontas ao longo do tempo, para esse exemplo.

Calculando o tempo médio de execução T_t e o tempo médio de espera T_w para a execução da Figura 6.2, temos:

$$T_t = \frac{t_t(t_1) + \cdots + t_t(t_5)}{5} = \frac{14 + 4 + 12 + 6 + 6}{5} = \frac{42}{5} = 8,4s$$

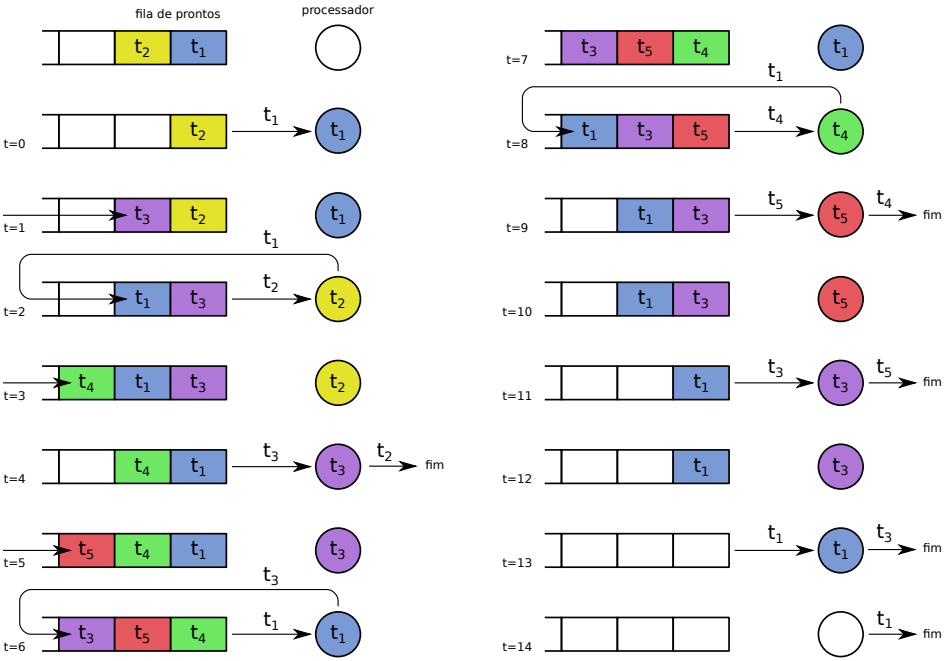


Figura 6.3: Evolução da fila de tarefas prontas no escalonamento *Round-Robin*.

$$T_w = \frac{t_w(t_1) + \cdots + t_w(t_5)}{5} = \frac{9+2+8+5+4}{5} = \frac{28}{5} = 5,6s$$

Observa-se o aumento nos tempos T_t e T_w em relação ao algoritmo FCFS simples, o que mostra que o algoritmo *Round-Robin* é menos eficiente para a execução de tarefas em lote. Entretanto, por distribuir melhor o uso do processador entre as tarefas ao longo do tempo, ele proporciona tempos de resposta bem melhores às aplicações interativas. Outro problema deste escalonador é o aumento no número de trocas de contexto, que, se for significativo, pode levar a uma degradação de desempenho, conforme discutido na Seção 5.2.

Deve-se observar que os algoritmos de escalonamento FCFS e RR não levam em conta a importância das tarefas nem seu comportamento em relação ao uso dos recursos. Por exemplo, nesses algoritmos as tarefas orientadas a entrada/saída irão receber menos tempo de processador que as tarefas orientadas a processamento (pois as primeiras geralmente não usam integralmente seus *quanta* de tempo), o que pode ser prejudicial para aplicações interativas.

6.4.3 Shortest Job First (SJF)

O algoritmo de escalonamento conhecido como *menor tarefa primeiro* (SJF - *Shortest Job First*) consiste em atribuir o processador à menor (mais curta) tarefa da fila de tarefas prontas. Esse algoritmo (e sua versão preemptiva, SRTF) proporciona os menores tempos

médios de espera das tarefas. Aplicando-se este algoritmo às tarefas da Tabela 6.1, obtém-se o escalonamento apresentado na Figura 6.4.

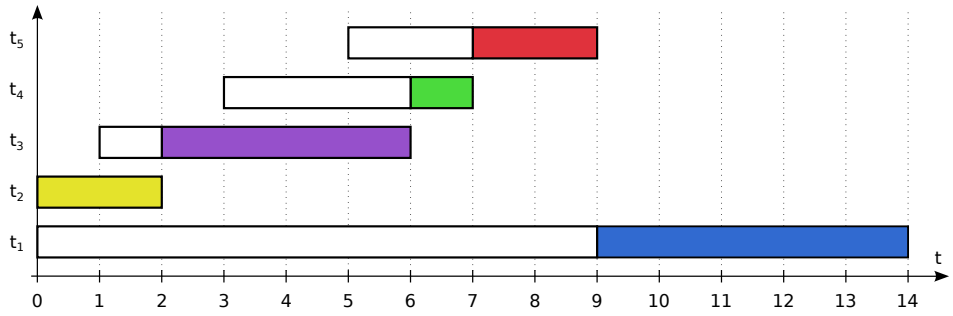


Figura 6.4: Escalonamento SJF.

Calculando o tempo médio de execução T_t e o tempo médio de espera T_w para a execução da Figura 6.4, temos:

$$T_t = \frac{t_t(t_1) + \cdots + t_t(t_5)}{5} = \frac{14 + 2 + 5 + 4 + 4}{5} = \frac{29}{5} = 5,8s$$

$$T_w = \frac{t_w(t_1) + \cdots + t_w(t_5)}{5} = \frac{9 + 0 + 1 + 3 + 2}{5} = \frac{15}{5} = 3,0s$$

A maior dificuldade no uso do algoritmo SJF consiste em estimar a priori a duração de cada tarefa, antes de sua execução. Com exceção de algumas tarefas em lote ou de tempo real, essa estimativa é inviável; por exemplo, como estimar por quanto tempo um editor de textos irá ser utilizado? Por causa desse problema, o algoritmo SJF puro é pouco utilizado. No entanto, ao associarmos o algoritmo SJF ao algoritmo RR, esse algoritmo pode ser de grande valia, sobretudo para tarefas orientadas a entrada/saída.

Supondo uma tarefa orientada a entrada/saída em um sistema preemptivo com $t_q = 10ms$. Nas últimas 3 vezes em que recebeu o processador, essa tarefa utilizou $3ms$, $4ms$ e $4,5ms$ de cada quantum recebido. Com base nesses dados históricos, é possível estimar qual a utilização de *quantum* provável da tarefa na próxima vez em que receber o processador. Essa estimativa pode ser feita por média simples (cálculo mais rápido) ou por extrapolação (cálculo mais complexo, podendo influenciar o tempo de troca de contexto t_{tc}).

A estimativa de uso do próximo quantum assim obtida pode ser usada como base para a aplicação do algoritmo SJF, o que irá beneficiar as tarefas orientadas a entrada/saída, que usam menos o processador. Obviamente, uma tarefa pode mudar de comportamento repentinamente, passando de uma fase de entrada/saída para uma fase de processamento, ou vice-versa. Nesse caso, a estimativa de uso do próximo *quantum* será incorreta durante alguns ciclos, mas logo voltará a refletir o comportamento atual da tarefa. Por essa razão, apenas a história recente da tarefa deve ser considerada (3 a 5 últimas ativações).

Outro problema associado ao escalonamento SJF é a possibilidade de *inanição* (*starvation*) das tarefas mais longas. Caso o fluxo de tarefas curtas chegando ao sistema

seja elevado, as tarefas mais longas nunca serão escolhidas para receber o processador e vão literalmente "morrer de fome", esperando na fila sem poder executar. Esse problema pode ser resolvido através de técnicas de envelhecimento de tarefas, como a apresentada na Seção 6.4.6.

6.4.4 Shortest Remaining Time First (SRTF)

O algoritmo SJF é cooperativo, ou seja, uma vez que uma tarefa recebe o processador, ela executa até encerrar (ou liberá-lo explicitamente). Em uma variante preemptiva, o escalonador deve comparar a duração prevista de cada nova tarefa que ingressa no sistema com o tempo de processamento restante das demais tarefas presentes, inclusive aquela que está executando no momento. Caso a nova tarefa tenha um tempo restante menor, ela recebe o processador. Essa abordagem é denominada por alguns autores de *menor tempo restante primeiro* (SRTF – *Short Remaining Time First*) [Tanenbaum, 2003]. O algoritmo SRTF está exemplificado na Figura 6.5.

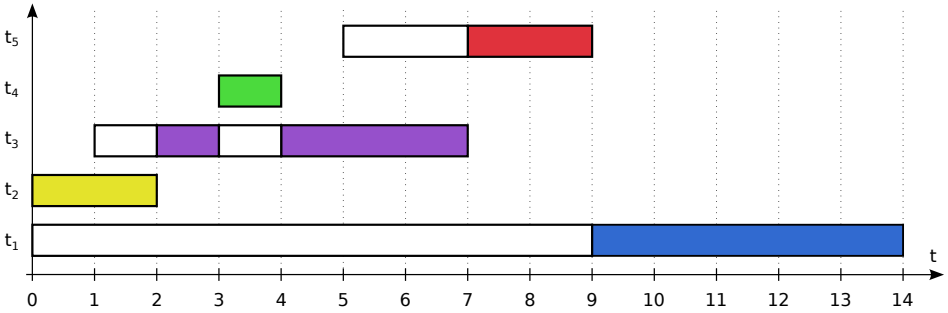


Figura 6.5: Escalonamento SRTF.

Calculando o tempo médio de execução T_t e o tempo médio de espera T_w para a execução da Figura 6.5, temos:

$$T_t = \frac{t_t(t_1) + \cdots + t_t(t_5)}{5} = \frac{14 + 2 + 6 + 1 + 4}{5} = \frac{27}{5} = 5,4s$$

$$T_w = \frac{t_w(t_1) + \cdots + t_w(t_5)}{5} = \frac{9 + 0 + 2 + 0 + 2}{5} = \frac{13}{5} = 2,6s$$

Pode-se observar que os tempos médios de execução T_t e de espera T_w são os menores observados até aqui. De fato, SRTF é o algoritmo de escalonamento que permite obter os mínimos valores de T_t e T_w. Ele torna o processamento de tarefas curtas muito eficiente, todavia com o risco de inanição das tarefas mais longas.

6.4.5 Escalonamento por prioridades fixas (PRIOc, PRIOp)

Vários critérios podem ser usados para ordenar a fila de tarefas prontas e escolher a próxima tarefa a executar; a data de ingresso da tarefa (usada no FCFS) e sua duração

prevista (usada no SJF) são apenas dois deles. Inúmeros outros critérios podem ser especificados, como o comportamento da tarefa (em lote, interativa ou de tempo real), seu proprietário (administrador, gerente, estagiário), seu grau de interatividade, etc.

No escalonamento por prioridade, a cada tarefa é associada uma prioridade, geralmente na forma de um número inteiro, que representa sua importância no sistema. Os valores de prioridade são então usados para definir a ordem de execução das tarefas. O algoritmo de escalonamento por prioridade define um modelo mais genérico de escalonamento, que permite modelar várias abordagens, entre as quais o FCFS e o SJF.

O escalonamento por prioridade pode ser cooperativo ou preemptivo. O diagrama da Figura 6.6 mostra o escalonamento das tarefas da Tabela 6.1 usando o **algoritmo por prioridade cooperativo**, ou PRIOc. Neste exemplo, os valores de prioridade são considerados em uma escala de prioridade positiva, ou seja, valores numéricos maiores indicam maior prioridade.

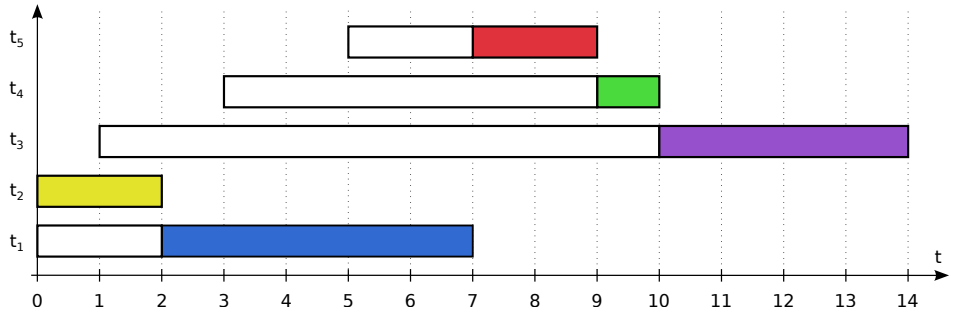


Figura 6.6: Escalonamento por prioridade cooperativo (PRIOc).

Calculando o tempo médio de execução T_t e o tempo médio de espera T_w para a execução da Figura 6.6, temos:

$$T_t = \frac{t_t(t_1) + \cdots + t_t(t_5)}{5} = \frac{7+2+13+7+4}{5} = \frac{33}{5} = 6,6s$$

$$T_w = \frac{t_w(t_1) + \cdots + t_w(t_5)}{5} = \frac{2+0+9+6+2}{5} = \frac{19}{5} = 3,8s$$

No **escalonamento por prioridade preemptivo** (PRIOp), quando uma tarefa de maior prioridade se torna disponível para execução, o escalonador entrega o processador a ela, trazendo a tarefa atualmente em execução de volta para a fila de prontas. Em outras palavras, a tarefa em execução pode ser "preemptada" por uma nova tarefa mais prioritária. Esse comportamento é apresentado na Figura 6.7 (observe que, quando t_4 ingressa no sistema, ela recebe o processador e t_1 volta a esperar na fila de prontas).

Calculando o tempo médio de execução T_t e o tempo médio de espera T_w para a execução da Figura 6.7, temos:

$$T_t = \frac{t_t(t_1) + \cdots + t_t(t_5)}{5} = \frac{10+2+13+1+2}{5} = \frac{28}{5} = 5,6s$$

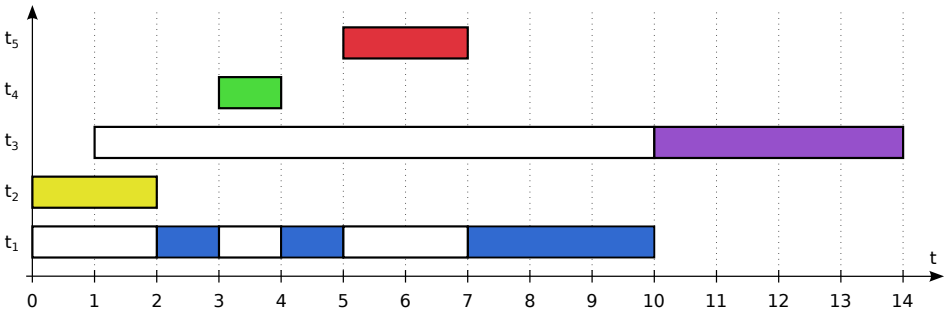


Figura 6.7: Escalonamento por prioridade preemptivo (PRIOp).

$$T_w = \frac{t_w(t_1) + \cdots + t_w(t_5)}{5} = \frac{5+0+9+0+0}{5} = \frac{14}{5} = 2,8s$$

6.4.6 Escalonamento por prioridades dinâmicas (PRIOd)

No escalonamento por prioridades fixas apresentado na seção anterior, as tarefas de menor prioridade só recebem o processador na ausência de tarefas de maior prioridade. Caso existam tarefas de maior prioridade frequentemente ativas, as de menor prioridade podem "morrer de fome" por nunca conseguir chegar ao processador. Esse fenômeno se denomina **inanição** (do inglês *starvation*).

Para evitar a inanição das tarefas de menor prioridade, um fator interno denominado **envelhecimento** (*aging*) deve ser definido. O envelhecimento aumenta a prioridade da tarefa proporcionalmente ao tempo que ela está aguardando o processador. Dessa forma, o envelhecimento define um esquema de **prioridades dinâmicas**, que permite a elas executar periodicamente e assim evitar a inanição.

Uma forma de implementar o envelhecimento de tarefas está indicada no algoritmo a seguir, que associa duas prioridades a cada tarefa t_i: a prioridade estática ou fixa pe_i, definida por fatores externos, e a prioridade dinâmica pd_i, que evolui ao longo da execução (considera-se uma escala de prioridades positiva).

Definições:
- N : número de tarefas no sistema
- t_i : tarefa i, $1 \leq i \leq N$
- pe_i : prioridade estática da tarefa t_i
- pd_i : prioridade dinâmica da tarefa t_i

Quando uma tarefa nova t_{nova} ingressa no sistema:
- $pe_{nova} \leftarrow$ *prioridade fixa*
- $pd_{nova} \leftarrow pe_{nova}$

Para escolher t_{prox}, a próxima tarefa a executar:

escolher $t_{prox} \mid pd_{prox} = max_{i=1}^{N}(pd_i)$
$\forall t_i \neq t_{prox} : pd_i \leftarrow pd_i + \alpha$
$pd_{prox} \leftarrow pe_{prox}$

O funcionamento é simples: a cada turno o escalonador escolhe como próxima tarefa (t_{prox}) aquela que tiver a maior prioridade dinâmica (pd). A prioridade dinâmica das demais tarefas é aumentada de uma constante α, conhecida como *fator de envelhecimento* (ou seja, essas tarefas "envelhecem" um pouco e no próximo turno terão mais chances de ser escolhidas). Por sua vez, a tarefa escolhida (t_{prox}) "rejuvenesce", tendo sua prioridade dinâmica ajustada de volta para o valor de sua prioridade estática ($pd_{prox} \leftarrow pe_{prox}$).

Aplicando o escalonamento com prioridades dinâmicas com envelhecimento, a execução das tarefas da Tabela 6.1 pode ser observada[2] na Figura 6.8.

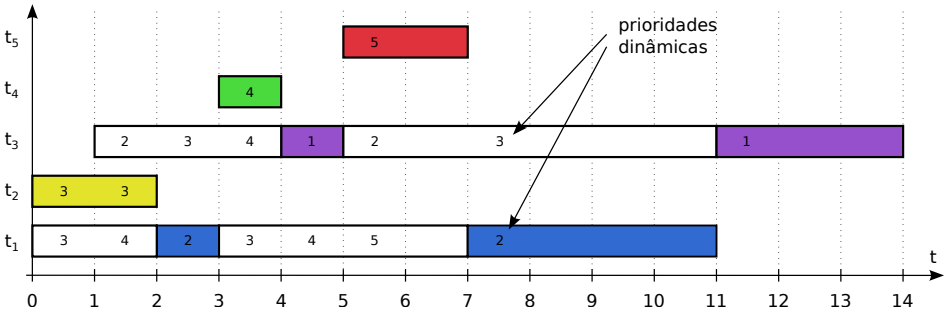


Figura 6.8: Escalonamento por prioridade preemptivo dinâmico (PRIOd).

Calculando o tempo médio de execução T_t e o tempo médio de espera T_w para a execução da Figura 6.8, temos:

$$T_t = \frac{t_t(t_1) + \cdots + t_t(t_5)}{5} = \frac{11 + 2 + 13 + 1 + 2}{5} = \frac{29}{5} = 5,8s$$

$$T_w = \frac{t_w(t_1) + \cdots + t_w(t_5)}{5} = \frac{6 + 0 + 9 + 0 + 0}{5} = \frac{15}{5} = 3,0s$$

As prioridades dinâmicas resolvem um problema importante em sistemas de tempo compartilhado. Nesses sistemas, as prioridades definidas pelo usuário estão intuitivamente relacionadas à **proporcionalidade** desejada na divisão do tempo de processamento. Por exemplo, caso um sistema de tempo compartilhado receba três tarefas: t_1 com prioridade 1, t_2 com prioridade 3 e t_3 com prioridade 6, espera-se que t_3 receba mais o processador que t_2, e esta mais que t_1 (assumindo uma escala de prioridades positiva).

Contudo, se associarmos apenas prioridades fixas a um escalonador *Round-Robin*, as tarefas irão executar de forma sequencial, sem a distribuição proporcional do processador.

[2]Observe que não ocorre envelhecimento em $t = 6$, pois esse exemplo trata de um sistema com preempção por prioridades, mas não por tempo. Por isso, as prioridades das tarefas só são reavaliadas quando uma tarefa inicia ou encerra. Assim, as prioridades das tarefas são reavaliadas em $t = 5$ (entrada de t_5) e $t = 7$ (conclusão de t_5), mas não em $t = 6$.

Esse resultado indesejável ocorre porque, a cada fim de *quantum*, sempre a tarefa mais prioritária é escolhida. Essa situação está ilustrada na Figura 6.9.

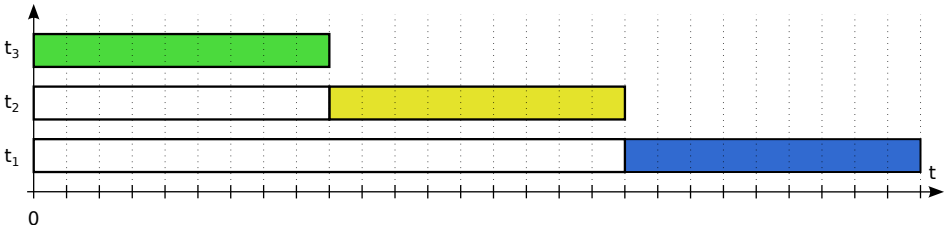


Figura 6.9: Escalonamento *Round-Robin* com prioridades fixas.

Usando o algoritmo de envelhecimento, a divisão do processador entre as tarefas se torna proporcional às suas prioridades estáticas. A Figura 6.10 ilustra a proporcionalidade obtida: percebe-se que todas as três tarefas recebem o processador periodicamente, mas que t_3 recebe mais tempo de processador que t_2, e que t_2 recebe mais que t_1.

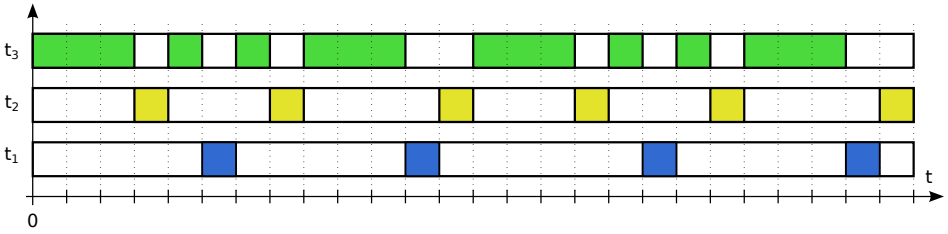


Figura 6.10: Escalonamento *Round-Robin* com prioridades dinâmicas.

6.4.7 Definição de prioridades

A definição da prioridade de uma tarefa é influenciada por diversos fatores, que podem ser classificados em dois grandes grupos:

Fatores externos: informações providas pelo usuário ou o administrador do sistema, que o escalonador não conseguiria estimar sozinho. Os fatores externos mais comuns são a classe do usuário (administrador, diretor, estagiário) o valor pago pelo uso do sistema (serviço básico, serviço *premium*) e a importância da tarefa em si (um detector de intrusão, um *script* de reconfiguração emergencial, etc.).

Fatores internos: informações que podem ser obtidas ou estimadas pelo próprio escalonador, com base em dados disponíveis no sistema operacional. Os fatores internos mais utilizados são a idade da tarefa, sua duração estimada, sua interatividade, seu uso de memória ou de outros recursos, seu envelhecimento, etc. Os fatores internos mudam continuamente e devem ser recalculados periodicamente pelo escalonador.

Os fatores externos são usualmente sintetizados em um valor inteiro denominado **prioridade estática** (ou *prioridade de base*), que resume a "opinião" do usuário ou administrador sobre a importância daquela tarefa. Em geral, cada família de sistemas operacionais define sua própria escala de prioridades estáticas. Alguns exemplos de escalas comuns são:

Windows (2000 em diante): processos e *threads* são associados a *classes de prioridade* (6 classes para processos e 7 classes para *threads*); a prioridade final de uma *thread* depende de sua prioridade de sua própria classe de prioridade e da classe de prioridade do processo ao qual está associada, assumindo valores entre 0 e 31. As prioridades do processos, apresentadas aos usuários no *Gerenciador de Tarefas*, apresentam os seguintes valores *default*:

- 24: *tempo real*
- 13: *alta*
- 10: *acima do normal*
- 8: *normal*
- 6: *abaixo do normal*
- 4: *baixa* ou *ociosa*

Além disso, geralmente a prioridade da tarefa responsável pela janela ativa recebe um incremento de prioridade: +1 ou +2, conforme a configuração do sistema (essa informação é considerada um fator interno de prioridade).

Linux (núcleo 2.4 em diante): considera duas escalas de prioridade separadas:

Tarefas de tempo real: usam uma escala de 1 a 99 positiva (valores maiores indicam **maior** prioridade). Somente o núcleo ou o administrador (*root*) podem lançar tarefas de tempo real.

Demais tarefas: usam uma escala que vai de −20 a +19, negativa (valores maiores indicam **menor** prioridade). Esta escala, denominada *nice level*, é padronizada em todos os sistemas UNIX-*like*. A prioridade das tarefas de usuário pode ser ajustada através dos comandos `nice` e `renice`.

6.4.8 Comparação entre os algoritmos apresentados

A Tabela 6.2 traz uma comparação sucinta entres os algoritmos apresentados nesta seção. Pode-se observar que os algoritmos preemptivos (RR, SRTF e PRIOp) possuem um número de trocas de contexto maior que seus correspondentes cooperativos, o que era de se esperar. Também pode-se constatar que o algoritmo SRTF proporciona os melhores tempos médios de execução T_t e de espera T_w, enquanto os piores tempos são providos pelo algoritmo RR (que, no entanto, oferece um melhor tempo de resposta a aplicações interativas).

Observa-se também que o tempo total de processamento é constante, pois ele só depende da carga de processamento de cada tarefa e não da ordem em que são executadas. Contudo, esse tempo pode ser influenciado pelo número de trocas de contexto, caso seja muito elevado.

Algoritmo de escalonamento	FCFS	RR	SJF	SRTF	PRIOc	PRIOp	PRIOd
Tempo médio de execução T_t	8,0	8,4	5,8	5,4	6,6	5,6	5,8
Tempo médio de espera T_w	5,2	5,6	3,0	2,6	3,8	2,8	3,0
Número de trocas de contexto	4	7	4	5	4	6	6
Tempo total de processamento	14	14	14	14	14	14	14

Tabela 6.2: Comparação entre os algoritmos apresentados.

6.4.9 Outros algoritmos de escalonamento

Além dos algoritmos vistos neste capítulo, diversos outros algoritmos foram propostos na literatura, alguns dos quais servem de base conceituais para escalonadores usados em sistemas operacionais reais. Dentre eles, podem ser citados os escalonadores de múltiplas filas, com ou sem *feedback* [Arpaci-Dusseau and Arpaci-Dusseau, 2014], os escalonadores justos [Kay and Lauder, 1988; Ford and Susarla, 1996], os escalonadores multiprocessador [Black, 1990] e multicore [Boyd-Wickizer et al., 2009], os escalonadores de tempo real [Farines et al., 2000] e os escalonadores multimídia [Nieh and Lam, 1997].

6.5 Escalonadores reais

Sistemas operacionais de mercado como Windows, Linux e MacOS são feitos para executar tarefas de diversos tipos, como jogos, editores de texto, conversores de vídeo, backups, etc. Para lidar com essa grande diversidade de tarefas, os escalonadores desses sistemas implementam algoritmos complexos, combinando mais de uma política de escalonamento.

No Linux, as tarefas são divididas em diversas classes de escalonamento, de acordo com suas demandas de processamento. Cada classe possui sua própria fila de tarefas, em um esquema conhecido como *Multiple Feedback Queues* [Arpaci-Dusseau and Arpaci-Dusseau, 2014]. As classes definidas no escalonador atual (núcleo 4.16, em 2018) são:

Classe `SCHED_DEADLINE`: classe específica para tarefas de tempo real que devem executar periodicamente e respeitar prazos (*deadlines*) predefinidos. A fila de tarefas desta classe é organizada por um algoritmo chamado *Earliest Deadline First* (EDF), usual em sistemas de tempo real [Farines et al., 2000].

Classe `SCHED_FIFO`: tarefas nesta classe são escalonadas usando uma política com prioridade fixa preemptiva, sem envelhecimento nem *quantum*. Portanto, uma tarefa nesta classe executa até bloquear por recursos, liberar explicitamente o processador (através da chamada de sistema `sched_yield()`) ou ser interrompida por outra tarefa de maior prioridade nesta mesma classe.

Classe `SCHED_RR`: esta classe implementa uma política similar à `SCHED_FIFO`, com a inclusão da preempção por tempo (*Round-Robin*). O valor default do *quantum* nesta classe é de 100 *ms*.

Classe `SCHED_NORMAL` ou `SCHED_OTHER`: é a classe padrão, que suporta as tarefas interativas dos usuários e a maioria dos serviços do SO. As tarefas são escalonadas por uma política baseada em prioridades dinâmicas (com envelhecimento) e *Round-Robin* com *quantum* variável, normalmente entre 0,75 *ms* e 6 *ms*.

Classe `SCHED_BATCH`: é similar à classe `SCHED_NORMAL`, mas pressupõe que as tarefas são orientadas a processamento (*CPU-bound*) e portanto são ativadas menos frequentemente que as tarefas em `SCHED_NORMAL`. Em compensação, recebem um *quantum* maior, de 1,5 segundos.

Classe `SCHED_IDLE`: classe com a menor prioridade de escalonamento; tarefas nesta classe só recebem processador caso não exista nenhuma outra tarefa em execução no sistema.

As tarefas são ativadas de acordo com sua classe, sendo `SCHED_DEADLINE` > `SCHED_FIFO/RR` > `SCHED_NORMAL` > `SCHED_BATCH` > `SCHED_IDLE`. Assim, tarefas nas chamadas classes interativas (`SCHED_NORMAL/BATCH/IDLE`) só executam se não houverem tarefas ativas nas classes denominadas de tempo real (`SCHED_DEADLINE/FIFO/RR`).

O núcleo Linux usa escalas de prioridade e algoritmos distintos para as várias classes de escalonamento. Para as tarefas interativas (`SCHED_NORMAL/BATCH/IDLE`) é usado o algoritmo *Completely Fair Scheduler* (CFS), baseado no conceito de escalonadores justos [Kay and Lauder, 1988], com prioridades entre -20 e +19. O CFS mantém a lista de tarefas prontas em uma árvore rubro-negra ordenada por tempos de processamento realizado, o que confere uma boa escalabilidade (os custos de busca, inserção e remoção nessa árvore são da ordem de $O(\log n)$). Por outro lado, as tarefas nas classes de tempo real têm prioridades entre 1 e 99 e são escalonadas por algoritmos específicos, como EDF (*Earliest Deadline First*) para `SCHED_DEADLINE`, o PRIOp para `SCHED_FIFO` e o PRIOp+RR para `SCHED_RR`.

A Tabela 6.3 sintetiza as principais características dos escalonadores disponíveis no núcleo Linux:

Classe	`DEADLINE`	`FIFO`	`RR`	`NORMAL`	`BATCH`	`IDLE`
Precedência	1	2	2	3	4	5
Algoritmo	EDF	PRIOp	PRIOp + Round Robin	CFS	CFS	CFS
Tempo real	✓	✓	✓			
Preempção por prioridade	✓	✓	✓	✓	✓	✓
Preempção por tempo			✓	✓	✓	✓
Quantum			100 *ms*	0,75 a 6 *ms*	1500 *ms*	1500 *ms*
Prioridade fixa	0	1 a 99	1 a 99	0	0	0
Níveis "nice"				-20 a +19	-20 a +19	> +19
Envelhecimento				✓	✓	

Tabela 6.3: Classes de escalonamento no núcleo Linux.

Exercícios

1. Explique o que é escalonamento *round-robin*, dando um exemplo.

2. Considere um sistema de tempo compartilhado com valor de quantum t_q e duração da troca de contexto t_{tc}. Considere tarefas de entrada/saída que usam em média p% de seu quantum de tempo cada vez que recebem o processador. Defina a eficiência $\mathcal{E}$ do sistema como uma função dos parâmetros t_q, t_{tc} e p.

3. Explique o que é, para que serve e como funciona a técnica de *aging*.

4. No algoritmo de envelhecimento definido na Seção 6.4.6, o que seria necessário modificar para suportar uma escala de prioridades negativa?

5. A tabela a seguir representa um conjunto de tarefas prontas para utilizar um processador:

Tarefa	t_1	t_2	t_3	t_4	t_5
ingresso	0	0	3	5	7
duração	5	4	5	6	4
prioridade	2	3	5	9	6

   Represente graficamente a sequência de execução das tarefas e calcule os tempos médios de vida (*tournaround time*) e de espera (*waiting time*), para as políticas de escalonamento a seguir:

   (a) FCFS cooperativa
   (b) SJF cooperativa
   (c) SJF preemptiva (SRTF)
   (d) PRIO cooperativa
   (e) PRIO preemptiva
   (f) RR com $t_q = 2$, sem envelhecimento

   Considerações: todas as tarefas são orientadas a processamento; as trocas de contexto têm duração nula; em eventuais empates (idade, prioridade, duração, etc), a tarefa t_i com menor i prevalece; valores maiores de prioridade indicam maior prioridade.

6. Idem, para as tarefas da tabela a seguir:

Tarefa	t_1	t_2	t_3	t_4	t_5
ingresso	0	0	1	7	11
duração	5	6	2	6	4
prioridade	2	3	4	7	9

Referências

D. Anderson, J. Cobb, E. Korpela, M. Lebofsky, and D. Werthimer. SETI@home: An experiment in public-resource computing. *Communications of the ACM*, 45(11):56–61, November 2002.

R. Arpaci-Dusseau and A. Arpaci-Dusseau. *Multi-level Feedback Queue*, chapter 8. Arpaci-Dusseau Books, 2014. http://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf.

D. L. Black. Scheduling and resource management techniques for multiprocessors. Technical Report CMU-CS-90-152, Carnegie-Mellon University, Computer Science Dept, 1990.

S. Boyd-Wickizer, R. Morris, and M. Kaashoek. Reinventing scheduling for multicore systems. In *12th conference on Hot topics in operating systems*, page 21. USENIX Association, 2009.

A. Burns and A. Wellings. *Real-Time Systems and Programming Languages, 2nd edition*. Addison-Wesley, 1997.

J.-M. Farines, J. da Silva Fraga, and R. S. de Oliveira. *Sistemas de Tempo Real – 12ª Escola de Computação da SBC*. Sociedade Brasileira de Computação, 2000. http://www.das.ufsc.br/gtr/livro.

B. Ford and S. Susarla. CPU Inheritance Scheduling. In *Usenix Association Second Symposium on Operating Systems Design and Implementation (OSDI)*, pages 91–105, 1996.

J. Kay and P. Lauder. A fair share scheduler. *Communications of the ACM*, 31(1):44–55, January 1988.

J. Nieh and M. Lam. The design, implementation and evaluation of SMART: a scheduler for multimedia applications. In *Proceedings of the 16th ACM Symposium on Operating Systems Principles*, pages 184–197, 1997.

A. Tanenbaum. *Sistemas Operacionais Modernos, 2ª edição*. Pearson – Prentice-Hall, 2003.

Capítulo 7

Tópicos em gestão de tarefas

Este capítulo traz tópicos de estudo específicos, que aprofundam ou complementam os temas apresentados nesta parte do livro, mas cuja leitura não é essencial para a compreensão do conteúdo principal da disciplina. Algumas seções deste capítulo podem estar incompletas ou não ter sido revisadas.

7.1 Inversão e herança de prioridades

Um problema importante que pode ocorrer em sistemas com escalonamento baseado em prioridade é a *inversão de prioridades* [Sha et al., 1990]. A inversão de prioridades ocorre quando uma tarefa de alta prioridade é impedida de executar por causa de uma tarefa de baixa prioridade.

Este tipo de problema envolve o conceito de *exclusão mútua*: alguns recursos do sistema devem ser usados por uma tarefa de cada vez, para evitar problemas de consistência de seu estado interno. Isso pode ocorrer com arquivos, portas de entrada/saída, áreas de memória compartilhada e conexões de rede, por exemplo. Quando uma tarefa obtém acesso a um recurso com exclusão mútua, as demais tarefas que desejam usá-lo ficam esperando no estado suspenso, até que o recurso esteja novamente livre. As técnicas usadas para implementar a exclusão mútua são descritas no Capítulo 10.

Para ilustrar esse problema, pode ser considerada a seguinte situação: um determinado sistema possui uma tarefa de alta prioridade t_a, uma tarefa de baixa prioridade t_b e uma tarefa de prioridade média t_m. Além disso, há um recurso R que deve ser acessado em exclusão mútua; para simplificar, somente t_a e t_b estão interessadas em usar esse recurso. A seguinte sequência de eventos, ilustrada na Figura 7.1, é um exemplo de como pode ocorrer uma inversão de prioridades:

1. Em um dado momento, o processador está livre e é alocado a uma tarefa de baixa prioridade t_b que é a única tarefa na fila de prontas;
2. durante seu processamento, t_b solicita acesso exclusivo ao recurso R e começa a usá-lo;
3. a tarefa t_m com prioridade maior que t_b acorda e volta à fila de prontas;

4. t_b volta ao final da fila de tarefas prontas, aguardando o processador; enquanto t_b não voltar a executar, o recurso R permanecerá alocado a ela e ninguém poderá usá-lo;

5. a tarefa de alta prioridade t_a acorda, volta à fila de prontas, recebe o processador e solicita acesso ao recurso R; como o recurso está alocado a t_b, a tarefa t_a é suspensa até que t_b libere o recurso.

Assim, a tarefa de alta prioridade t_a não pode executar, porque o recurso de que necessita está nas mãos da tarefa de baixa prioridade t_b. Por sua vez, t_b não pode executar, pois o processador está ocupado com t_m.

Dessa forma, t_a deve esperar que t_b execute e libere R, o que configura a inversão de prioridades. A espera de t_a pode ser longa, pois t_b tem baixa prioridade e pode demorar a executar novamente, caso existam outras tarefas em execução no sistema (como t_m). Como tarefas de alta prioridade são geralmente críticas para o funcionamento de um sistema, a inversão de prioridades pode ter efeitos graves.

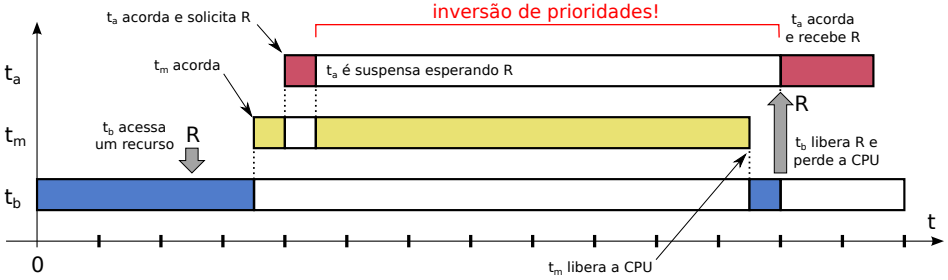


Figura 7.1: Cenário de uma inversão de prioridades.

Uma solução elegante para o problema da inversão de prioridades é obtida através de um *protocolo de herança de prioridade* [Sha et al., 1990]. O protocolo de herança de prioridade mais simples consiste em aumentar temporariamente a prioridade da tarefa t_b que detém o recurso de uso exclusivo R. Caso esse recurso seja requisitado por uma tarefa de maior prioridade t_a, a tarefa t_b "herda" temporariamente a prioridade de t_a, para que possa executar e liberar o recurso R mais rapidamente. Assim que liberar o recurso, t_b retorna à sua prioridade anterior. Essa estratégia está ilustrada na Figura 7.2.

Provavelmente o mais célebre exemplo real de inversão de prioridades tenha ocorrido na sonda espacial *Mars Pathfinder*, enviada pela NASA em 1996 para explorar o solo do planeta Marte (Figura 7.3) [Jones, 1997]. O software da sonda executava sobre o sistema operacional de tempo real *VxWorks* e consistia de 97 *threads* com vários níveis de prioridades fixas. Essas tarefas se comunicavam através de uma área de transferência em memória compartilhada (na verdade, um *pipe* UNIX), com acesso mutuamente exclusivo controlado por semáforos (semáforos são estruturas de sincronização discutidas na Seção 11.1).

A gerência da área de transferência estava a cargo de uma tarefa t_{ger}, rápida e de alta prioridade, que era ativada frequentemente para mover blocos de informação para dentro e fora dessa área. A coleta de dados meteorológicos era feita por uma tarefa t_{met}

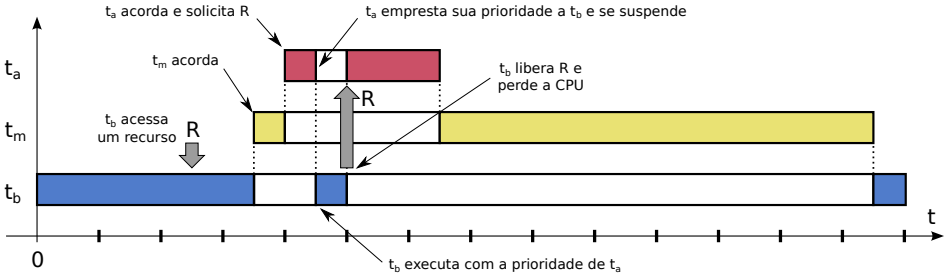


Figura 7.2: Um protocolo de herança de prioridade.

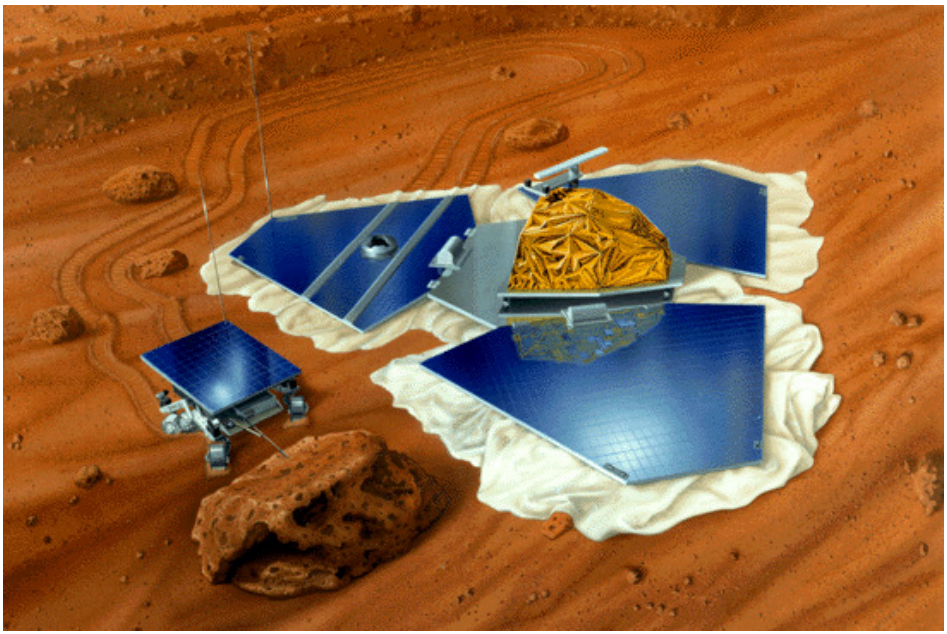


Figura 7.3: Sonda *Mars Pathfinder* com o robô *Sojourner* (NASA).

de baixa prioridade, que executava esporadicamente e escrevia seus dados na área de transferência, para uso por outras tarefas. Por fim, a comunicação com a Terra estava sob a responsabilidade de uma tarefa t_{com} de prioridade média e potencialmente demorada (Tabela 7.1 e Figura 7.4).

Como o sistema *VxWorks* usa um escalonador preemptivo com prioridades fixas, as tarefas eram atendidas conforme suas necessidades na maior parte do tempo. Todavia, a exclusão mútua no acesso à área de transferência escondia uma inversão de prioridades: caso a tarefa de coleta de dados meteorológicos t_{met} perdesse o processador sem liberar a área de transferência, a tarefa de gerência t_{ger} teria de ficar esperando até que t_{met} voltasse a executar para liberar a área. Isso poderia demorar se, por azar, a tarefa de comunicação t_{com} estivesse executando, pois ela tinha mais prioridade que t_{met}.

tarefa	função	prioridade	duração
t_{ger}	gerência da área de transferência	alta	curta
t_{met}	coleta de dados meteorológicos	baixa	curta
t_{com}	comunicação com a Terra	média	longa

Tabela 7.1: Algumas tarefas do software da sonda *Mars Pathfinder*.

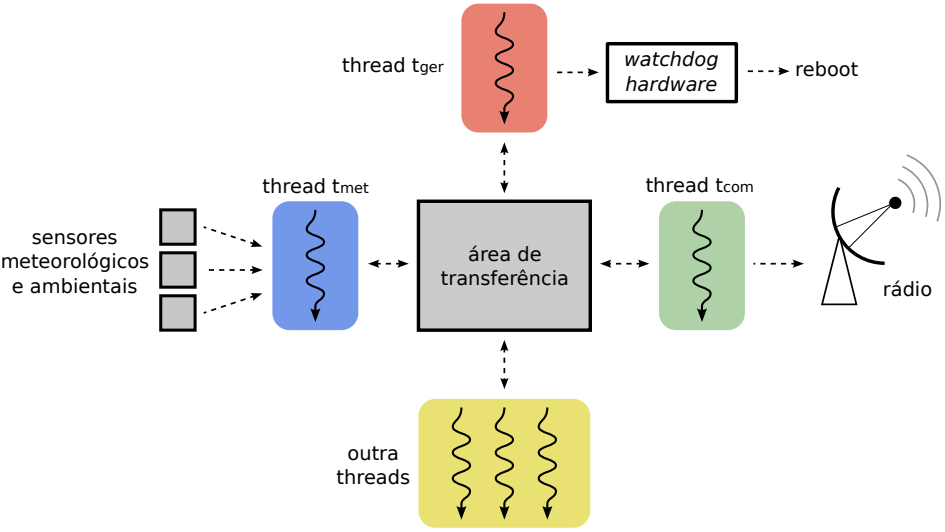


Figura 7.4: Principais tarefas do software embarcado da sonda *Mars Pathfinder*.

Como todos os sistemas críticos, a sonda *Mars Pathfinder* possui um sistema de proteção contra erros, ativado por um temporizador (*watchdog*). Caso a gerência da área de transferência ficasse parada por muito tempo, um procedimento de reinício geral do sistema (*reboot*) era automaticamente ativado pelo temporizador. Dessa forma, a inversão de prioridades provocava reinícios esporádicos e imprevisíveis no software da sonda, interrompendo suas atividades e prejudicando seu funcionamento. A solução foi obtida através de um *patch*[1] que ativou a herança de prioridades: caso a tarefa de gerência t_{ger} fosse bloqueada pela tarefa de coleta de dados t_{met}, esta última herdava a alta prioridade de t_{ger} para poder liberar rapidamente a área de transferência, mesmo se a tarefa de comunicação t_{com} estivesse em execução.

Exercícios

1. Explique os conceitos de *inversão* e *herança de prioridade*.

[1] Fica ao leitor imaginar como pode ser depurado e corrigido um bug de software em uma sonda a 100 milhões de Km da Terra...

2. O sistema operacional da sonda *Mars Pathfinder* (Seção 7.1) usa escalonamento por prioridades preemptivo, sem envelhecimento e sem compartilhamento de tempo. Suponha que as tarefas t_g e t_m detêm a área de transferência de dados durante todo o período em que executam. Os dados de um trecho de execução das tarefas são indicados na tabela a seguir (observe que t_g executa mais de uma vez).

Tarefa	t_g	t_m	t_c
ingresso	0, 5, 10	2	3
duração	1	2	10
prioridade	alta	baixa	média

Desenhe o diagrama de tempo da execução sem e com o protocolo de herança de prioridades e discuta sobre as diferenças observadas entre as duas execuções.

Referências

M. Jones. What really happened on Mars Rover Pathfinder. *ACM Risks-Forum Digest*, 19 (49), 1997.

L. Sha, R. Rajkumar, and J. Lehoczky. Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on Computers*, 39(9):1175–1185, 1990.